Лист слепой проверки (B, C, D)

Собрано из файлов проекта командой `.venv/bin/python -m src.human_docs` · коммит ac61927

> **Только для чтения.** Оценки вносятся в CSV: проверяющий 1 — `grading/sheet_r1.csv`, проверяющий 2 — `grading/sheet_r2.csv`. Порядок строк одинаковый. Модель-автор скрыта. Не обсуждайте оценки друг с другом и не открывайте `grading/key_DO_NOT_OPEN.csv`.

Ответов: 80 (B — 32, C — 24, D — 24).

B001 · тип B

Задача: Верно или неверно утверждение: «Если в логистической регрессии увеличить порог с 0.5 до 0.7, вероятности, которые выдаёт модель, не изменятся, но объектов, отнесённых к положительному классу, может стать меньше»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
- логистическая регрессия сначала вычисляет вероятность принадлежности объекта к положительному
классу
- порог используется только для перевода вероятности в метку класса, поэтому его изменение не
влияет на сами вероятности
- при увеличении порога часть объектов, которые раньше относились к положительному классу,
может перейти в отрицательный

Типичные ошибки:
- утверждение, что изменение порога меняет вероятности, рассчитанные моделью
- путаница между вероятностью и предсказанной меткой класса
- утверждение, что увеличение порога обязательно изменит количество положительных предсказаний

2 — вердикт верный и обоснование содержит ключевые аргументы
1 — вердикт верный, но обоснование неполное или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Верно.

Когда порог увеличивается с 0.5 до 0.7 в логистической регрессии, вероятности, предсказанные
моделью, остаются неизменными, так как они зависят только от модели и не зависят от порога.
Однако, при увеличении порога модель будет классифицировать объекты как положительные только
тогда, когда их вероятность превышает 0.7, что приведёт к уменьшению количества объектов,
отнесённых к положительному классу.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B002 · тип B

Задача: Верно или неверно утверждение: «Случайный лес не требует масштабирования признаков, а SVM с RBF-ядром требует»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– деревья разбивают по порогу одного признака: монотонное преобразование признака не меняет
разбиения
– RBF-ядро зависит от евклидова расстояния: признак с большим масштабом доминирует, gamma
теряет смысл

Типичные ошибки:
– ответ «неверно», потому что «масштабирование нужно всегда»
– утверждать, что случайному лесу масштабирование заметно помогает

2 — «верно» и названы обе причины (пороги в деревьях, расстояния в RBF)
1 — «верно», но объяснена только одна часть или есть ошибка в объяснении
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение верно.

Обоснование:

1. Случайный лес основан на ансамбле деревьев решений. Деревья решений работают непосредственно
с признаками, используя пороговые значения для разделения данных. Масштабирование признаков не
влияет на работу алгоритма, поскольку деревья решений используют ранжированные сравнения, а не
абсолютные расстояния между значениями признаков.

2. SVM с RBF-ядром (радиально-базисным ядром) чувствителен к масштабу признаков. Ядро RBF
основано на евклидовом расстоянии между точками, которое зависит от масштаба признаков. Без
предварительного масштабирования признаки с большим диапазоном значений будут доминировать над
признаками с меньшим диапазоном, что приведет к смещению модели и ухудшению качества
классификации или регрессии.

Таким образом, случайный лес действительно не требует масштабирования признаков, тогда как SVM
с RBF-ядром требует.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B003 · тип B

Задача: Верно или неверно утверждение: «Наивный байесовский классификатор даёт хорошо откалиброванные вероятности, потому что он вероятностный»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
– предположение об условной независимости признаков обычно нарушено: зависимые признаки
учитываются многократно, вероятности смещаются к 0 и 1 (переуверенность)
– вероятностная модель не означает калиброванных вероятностей; калибровку проверяют диаграммой
надёжности
– исправляется калибровкой (Platt, isotonic, CalibratedClassifierCV)

Типичные ошибки:
```

```
- ответ «верно», раз модель «выдаёт вероятности»
- путать хорошую точность классификации с хорошей калибровкой

2 — «неверно» и названа причина (нарушение независимости → переуверенные вероятности)
1 — «неверно», но причина не названа или объяснена с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Неверно. Наивный байесовский классификатор не даёт хорошо откалиброванные вероятности, несмотря
на то, что он вероятностный. Это связано с тем, что он делает сильные предположения о
независимости признаков, которые редко выполняются на практике. Эти предположения могут
привести к искажению оценок вероятностей, что делает их менее откалиброванными по сравнению с
другими методами, такими как логистическая регрессия или деревья решений с калибровкой.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B004 · тип B

Задача: Верно или неверно утверждение: «PCA выбирает направления, лучше всего разделяющие классы»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
- PCA — метод без учителя: метки классов не используются
- PCA ищет направления максимальной дисперсии; они могут не разделять классы
- направления, разделяющие классы, ищет LDA (линейный дискриминант Фишера)

Типичные ошибки:
- ответ «верно», путая PCA и LDA
- считать, что большая дисперсия всегда означает лучшее разделение

2 — «неверно» + хотя бы один из доводов: PCA не использует метки или максимизирует дисперсию, а
не разделимость
1 — «неверно», но обоснование с ошибкой или без этих доводов
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение неверно. Метод главных компонент (PCA) не ориентирован на разделение классов. Его
основная цель — уменьшить размерность данных, сохранив при этом максимально возможную
вариативность. PCA находит направления (главные компоненты), вдоль которых данные имеют
наибольшую дисперсию, а не те, которые лучше всего разделяют классы. Для разделения классов
чаще используются другие методы, например, линейный дискриминантный анализ (LDA).

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B005 · тип B

Задача: Верно или неверно утверждение: «Если в логистической регрессии увеличить порог с 0.5 до 0.7, вероятности, которые выдаёт модель, не изменятся, но объектов, отнесённых к положительному классу, может стать меньше»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– логистическая регрессия сначала вычисляет вероятность принадлежности объекта к положительному классу
– порог используется только для перевода вероятности в метку класса, поэтому его изменение не влияет на сами вероятности
– при увеличении порога часть объектов, которые раньше относились к положительному классу, может перейти в отрицательный

Типичные ошибки:
– утверждение, что изменение порога меняет вероятности, рассчитанные моделью
– путаница между вероятностью и предсказанной меткой класса
– утверждение, что увеличение порога обязательно изменит количество положительных предсказаний

2 — вердикт верный и обоснование содержит ключевые аргументы
1 — вердикт верный, но обоснование неполное или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение верно.

Логистическая регрессия предсказывает вероятность принадлежности объекта к положительному классу. Пороговое значение используется для преобразования этих вероятностей в бинарную классификацию. По умолчанию этот порог часто устанавливается равным 0.5, что означает, что объект относится к положительному классу, если прогнозируемая вероятность больше или равна 0.5.

Если мы увеличим порог до 0.7, то теперь объект будет относиться к положительному классу только тогда, когда прогнозируемая вероятность превышает 0.7. Это приведет к тому, что меньшее количество объектов будет отнесено к положительному классу, поскольку теперь требуется более высокая уверенность модели в том, что объект принадлежит к этому классу.

Сам алгоритм логистической регрессии и его параметры остаются неизменными; меняются лишь критерии принятия решений относительно классификации объектов.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B006 · тип B

Задача: Верно или неверно утверждение: «Наивный байесовский классификатор даёт хорошо откалиброванные вероятности, потому что он вероятностный»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
– предположение об условной независимости признаков обычно нарушено: зависимые признаки учитываются многократно, вероятности смещаются к 0 и 1 (переуверенность)
– вероятностная модель не означает калиброванных вероятностей; калибровку проверяют диаграммой
```

```
надёжности
– исправляется калибровкой (Platt, isotonic, CalibratedClassifierCV)

Типичные ошибки:
– ответ «верно», раз модель «выдаёт вероятности»
– путать хорошую точность классификации с хорошей калибровкой

2 — «неверно» и названа причина (нарушение независимости → переуверенные вероятности)
1 — «неверно», но причина не названа или объяснена с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Наивный байесовский классификатор действительно является вероятностной моделью, так как он
основан на применении теоремы Байеса для вычисления апостериорных вероятностей классов. Однако
утверждение, что он даёт хорошо откалиброванные вероятности, не всегда верно. Калибровка
вероятностей зависит от ряда факторов, включая распределение данных и независимость признаков
(что часто нарушается в реальных задачах). На практике наивный байесовский классификатор
нередко демонстрирует недостаточное качество калибровки вероятностей, особенно когда нарушается
предположение о независимости признаков.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B007 · тип B

Задача: Верно или неверно утверждение: «Если добавить новый признак в линейную регрессию без регуляризации и заново обучить модель методом МНК, ошибка на обучающих данных не увеличится, даже если новый признак бесполезный»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– МНК подбирает коэффициенты так, чтобы минимизировать сумму квадратов ошибок
– если новый признак не помогает, модель может просто поставить его коэффициент равным нулю
– поэтому минимальная ошибка MSE на обучающих данных не может увеличиться, но на тестовых
данных качество может ухудшиться

Типичные ошибки:
– утверждение, что бесполезный признак обязательно увеличит ошибку на обучении
– путаница между ошибкой на обучающих и тестовых данных
– утверждение, что добавление признака всегда улучшает качество модели

2 — «верно» и объяснено, что старое решение с нулевым коэффициентом при новом признаке остаётся
допустимым, поэтому минимум ошибки на обучении не растёт
1 — «верно», но обоснование неполное (нет аргумента про нулевой коэффициент / вложенность
моделей) или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Добавление нового бесполезного признака в линейную регрессию без регуляризации и обучение
модели методом МНК может привести к увеличению ошибки на обучающих данных. Это связано с тем,
что модель будет пытаться подогнать параметры под шум или случайные колебания в данных, что
может ухудшить качество предсказаний. Новый признак, не содержащий полезной информации, может
вызвать переобучение, особенно если он коррелирует с другими признаками или является полностью
```

```
случайным. Таким образом, утверждение неверно.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B008 · тип B

Задача: Верно или неверно утверждение: «Случайный лес не требует масштабирования признаков, а SVM с RBF-ядром требует»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– деревья разбивают по порогу одного признака: монотонное преобразование признака не меняет разбиения
– RBF-ядро зависит от евклидова расстояния: признак с большим масштабом доминирует, gamma теряет смысл

Типичные ошибки:
– ответ «неверно», потому что «масштабирование нужно всегда»
– утверждать, что случайному лесу масштабирование заметно помогает

2 — «верно» и названы обе причины (пороги в деревьях, расстояния в RBF)
1 — «верно», но объяснена только одна часть или есть ошибка в объяснении
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Случайный лес не требует масштабирования признаков, так как он строит деревья на основе признаков без учета их масштаба. В то же время, SVM с RBF-ядром чувствителен к масштабу признаков, так как расстояние между точками влияет на работу ядра. Поэтому утверждение верно.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B009 · тип B

Задача: Верно или неверно утверждение: «Алгоритм кластеризации с более высоким средним коэффициентом силуэта может хуже восстанавливать истинную структуру групп в данных, чем алгоритм с более низким коэффициентом силуэта»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– коэффициент силуэта — внутренняя метрика: оценивает компактность кластеров и их отделённость друг от друга, истинные метки не использует
– высокий силуэт не гарантирует соответствия кластеров истинным группам объектов
– метрика предпочитает компактные (выпуклые) кластеры, даже если реальные группы имеют сложную геометрическую форму
```

```
– пример: k-means может разрезать кольцеобразный кластер на части и получить силуэт выше, чем
HDBSCAN, который корректно выделит исходную структуру

Типичные ошибки:
– утверждение, что максимальный коэффициент силуэта всегда соответствует наилучшей
кластеризации
– путаница между внутренними и внешними метриками качества кластеризации
– предположение, что коэффициент силуэта непосредственно измеряет соответствие истинным меткам
– игнорирование влияния формы кластеров на оценку качества

2 — правильно определена истинность утверждения («верно»), объяснено ограничение коэффициента
силуэта и приведён пример
1 — правильно определена истинность, но обоснование неполное
0 — неверный ответ или некорректное обоснование
```

Ответ модели:

```
Коэффициент силуэта используется для оценки качества кластеризации и показывает, насколько
хорошо объекты группируются в свои кластеры по сравнению с другими кластерами. Значение
коэффициента варьируется от −1 до 1, где близкие к 1 значения указывают на хорошую
кластеризацию, а близкие к −1 — на плохую. Однако коэффициент силуэта не всегда даёт полную
картину. Например, в некоторых случаях алгоритм с более высоким коэффициентом силуэта может не
учитывать сложную структуру данных и объединять разнородные объекты в один кластер, в то время
как алгоритм с более низким коэффициентом может лучше отражать истинную структуру групп, даже
если его кластеры менее «чистые» с точки зрения коэффициента силуэта. Таким образом,
утверждение, что алгоритм с более высоким средним коэффициентом силуэта может хуже
восстанавливать истинную структуру групп в данных, чем алгоритм с более низким коэффициентом
силуэта, является верным.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B010 · тип B

Задача: Верно или неверно утверждение: «PCA выбирает направления, лучше всего разделяющие классы»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
– PCA — метод без учителя: метки классов не используются
– PCA ищет направления максимальной дисперсии; они могут не разделять классы
– направления, разделяющие классы, ищет LDA (линейный дискриминант Фишера)

Типичные ошибки:
– ответ «верно», путая PCA и LDA
– считать, что большая дисперсия всегда означает лучшее разделение

2 — «неверно» + хотя бы один из доводов: PCA не использует метки или максимизирует дисперсию, а
не разделимость
1 — «неверно», но обоснование с ошибкой или без этих доводов
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение **неверно**.
PCA (метод главных компонент) выбирает направления, вдоль которых **дисперсия данных
максимальна**, а не те, которые лучше всего разделяют классы.
PCA не использует информацию о метках классов, поэтому он может выбрать направление, которое
вообще не помогает разделению классов, если в этом направлении велика внутриклассовая или общая
дисперсия.
Для разделения классов предназначены другие методы, например **LDA (линейный дискриминантный
анализ)**, который как раз ищет направления, максимизирующие межклассовую разделимость.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B011 · тип B

Задача: Верно или неверно утверждение: «Алгоритм кластеризации с более высоким средним коэффициентом силуэта может хуже восстанавливать истинную структуру групп в данных, чем алгоритм с более низким коэффициентом силуэта»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– коэффициент силуэта — внутренняя метрика: оценивает компактность кластеров и их отделённость
друг от друга, истинные метки не использует
– высокий силуэт не гарантирует соответствия кластеров истинным группам объектов
– метрика предпочитает компактные (выпуклые) кластеры, даже если реальные группы имеют сложную
геометрическую форму
– пример: k-means может разрезать кольцеобразный кластер на части и получить силуэт выше, чем
HDBSCAN, который корректно выделит исходную структуру

Типичные ошибки:
– утверждение, что максимальный коэффициент силуэта всегда соответствует наилучшей
кластеризации
– путаница между внутренними и внешними метриками качества кластеризации
– предположение, что коэффициент силуэта непосредственно измеряет соответствие истинным меткам
– игнорирование влияния формы кластеров на оценку качества

2 — правильно определена истинность утверждения («верно»), объяснено ограничение коэффициента
силуэта и приведён пример
1 — правильно определена истинность, но обоснование неполное
0 — неверный ответ или некорректное обоснование
```

Ответ модели:

```
Утверждение **верно**.

Средний коэффициент силуэта измеряет, насколько компактны и хорошо разделены кластеры с точки
зрения выбранной метрики расстояния, но он не учитывает, соответствует ли полученное разбиение
истинной структуре групп. Возможны ситуации, когда алгоритм с более высоким силуэтом выделяет
геометрически «удобные» кластеры, которые не совпадают с реальными классами, тогда как алгоритм
с более низким силуэтом лучше восстанавливает истинные группы, но из-за их формы, перекрытия
или шума имеет худшую оценку силуэта.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B012 · тип B

Задача: Верно или неверно утверждение: «Наивный байесовский классификатор даёт хорошо откалиброванные вероятности, потому что он вероятностный»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
- предположение об условной независимости признаков обычно нарушено: зависимые признаки учитываются многократно, вероятности смещаются к 0 и 1 (переуверенность)
- вероятностная модель не означает калиброванных вероятностей; калибровку проверяют диаграммой надёжности
- исправляется калибровкой (Platt, isotonic, CalibratedClassifierCV)

Типичные ошибки:
- ответ «верно», раз модель «выдаёт вероятности»
- путать хорошую точность классификации с хорошей калибровкой

2 — «неверно» и названа причина (нарушение независимости → переуверенные вероятности)
1 — «неверно», но причина не названа или объяснена с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение неверно. Хотя наивный байесовский классификатор является вероятностным методом, основанным на теореме Байеса, он часто не обеспечивает хорошо откалиброванные вероятности. Это связано с предположением о независимости признаков, которое редко выполняется на практике. В результате полученные вероятности могут быть смещёнными или чрезмерно уверенными.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 90
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B013 · тип B

Задача: Верно или неверно утверждение: «Если у модели вырос F1-score, значит и precision, и recall тоже обязательно стали выше»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
- F1-score — это гармоническое среднее precision и recall
- F1 может увеличиться, даже если одна из двух метрик уменьшилась, а другая выросла достаточно сильно
- например, при precision = 0.8 и recall = 0.4 значение F1 примерно равно 0.53, а при precision = 0.7 и recall = 0.6 оно увеличится до 0.65, хотя precision снизился

Типичные ошибки:
- утверждение, что рост F1 всегда означает рост обеих метрик
- путаница между F1-score, precision и recall
```

```
- непонимание того, что увеличение одной метрики может компенсировать снижение другой

2 — вердикт верный и обоснование содержит ключевые аргументы
1 — вердикт верный, но обоснование неполное или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение **неверно**.

F1-score — это среднее гармоническое между precision и recall:

\[
F_1 = 2 \cdot \frac{P \cdot R}{P + R}
\]

Он может вырасти даже в том случае, если одна из метрик немного снизилась, но другая выросла значительно. Например, если precision упал с 0.90 до 0.85, а recall вырос с 0.50 до 0.80, то F1 увеличится, несмотря на снижение precision.
Поэтому рост F1-score не гарантирует одновременного роста обеих метрик.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B014 · тип B

Задача: Верно или неверно утверждение: «PCA выбирает направления, лучше всего разделяющие классы»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
- PCA — метод без учителя: метки классов не используются
- PCA ищет направления максимальной дисперсии; они могут не разделять классы
- направления, разделяющие классы, ищет LDA (линейный дискриминант Фишера)

Типичные ошибки:
- ответ «верно», путая PCA и LDA
- считать, что большая дисперсия всегда означает лучшее разделение

2 — «неверно» + хотя бы один из доводов: PCA не использует метки или максимизирует дисперсию, а не разделимость
1 — «неверно», но обоснование с ошибкой или без этих доводов
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
PCA (Principal Component Analysis) — это метод снижения размерности, который ищет направления (компоненты), вдоль которых данные имеют наибольшую дисперсию. Однако PCA не учитывает информацию о классах и не стремится к разделяемости классов. Он полностью основан на статистических свойствах данных, а не на их классовом разнообразии.

Таким образом, утверждение «PCA выбирает направления, лучше всего разделяющие классы» является **неверным**.
```

```
ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B015 · тип B

Задача: Верно или неверно утверждение: «Если в логистической регрессии увеличить порог с 0.5 до 0.7, вероятности, которые выдаёт модель, не изменятся, но объектов, отнесённых к положительному классу, может стать меньше»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– логистическая регрессия сначала вычисляет вероятность принадлежности объекта к положительному классу
– порог используется только для перевода вероятности в метку класса, поэтому его изменение не влияет на сами вероятности
– при увеличении порога часть объектов, которые раньше относились к положительному классу, может перейти в отрицательный

Типичные ошибки:
– утверждение, что изменение порога меняет вероятности, рассчитанные моделью
– путаница между вероятностью и предсказанной меткой класса
– утверждение, что увеличение порога обязательно изменит количество положительных предсказаний

2 – вердикт верный и обоснование содержит ключевые аргументы
1 – вердикт верный, но обоснование неполное или с ошибкой
0 – вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение **верно**.

Логистическая регрессия выдаёт для каждого объекта вероятность принадлежности к положительному классу \( P(y=1|x) \). Эта вероятность вычисляется на основе обученных коэффициентов модели и не зависит от выбранного порога классификации.

Порог используется только на этапе принятия решения: объект относят к положительному классу, если \( P(y=1|x) \ge \text{threshold} \).
Если порог увеличить с 0.5 до 0.7, то часть объектов, у которых вероятность находится в диапазоне \([0.5, 0.7)\), перестанут классифицироваться как положительные, поэтому количество объектов, отнесённых к положительному классу, может уменьшиться. Сами же вероятности, выдаваемые моделью, останутся прежними.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B016 · тип B

Задача: Верно или неверно утверждение: «Если добавить новый признак в линейную регрессию без регуляризации и заново обучить модель методом МНК, ошибка на обучающих данных не увеличится, даже если новый признак бесполезный»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– МНК подбирает коэффициенты так, чтобы минимизировать сумму квадратов ошибок
– если новый признак не помогает, модель может просто поставить его коэффициент равным нулю
– поэтому минимальная ошибка MSE на обучающих данных не может увеличиться, но на тестовых
данных качество может ухудшиться

Типичные ошибки:
– утверждение, что бесполезный признак обязательно увеличит ошибку на обучении
– путаница между ошибкой на обучающих и тестовых данных
– утверждение, что добавление признака всегда улучшает качество модели

2 – «верно» и объяснено, что старое решение с нулевым коэффициентом при новом признаке остаётся
допустимым, поэтому минимум ошибки на обучении не растёт
1 – «верно», но обоснование неполное (нет аргумента про нулевой коэффициент / вложенность
моделей) или с ошибкой
0 – вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение **верно**.

Обоснование:
В линейной регрессии без регуляризации обучение методом наименьших квадратов (МНК) минимизирует
сумму квадратов ошибок на обучающей выборке. Если добавить новый признак, модель получает
больше степеней свободы – она может, как минимум, присвоить новому признаку нулевой коэффициент
и получить ровно ту же ошибку, что и раньше. Но поскольку МНК ищет глобальный минимум ошибки,
добавление признака не может ухудшить результат на обучающих данных – в худшем случае ошибка
останется прежней, а в общем случае уменьшится. Это верно даже для бесполезного признака, если
только он не вызывает численных проблем вырожденности (но в теории, при точных вычислениях,
ошибка не возрастёт).

ОТВЕТ: верно
УВЕРЕННОСТЬ: 99
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B017 · тип B

Задача: Верно или неверно утверждение: «Если у модели вырос F1-score, значит и precision, и recall тоже обязательно стали выше»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
– F1-score – это гармоническое среднее precision и recall
– F1 может увеличиться, даже если одна из двух метрик уменьшилась, а другая выросла достаточно
сильно
– например, при precision = 0.8 и recall = 0.4 значение F1 примерно равно 0.53, а при precision
= 0.7 и recall = 0.6 оно увеличится до 0.65, хотя precision снизился

Типичные ошибки:
– утверждение, что рост F1 всегда означает рост обеих метрик
– путаница между F1-score, precision и recall
– непонимание того, что увеличение одной метрики может компенсировать снижение другой
```

```
2 — вердикт верный и обоснование содержит ключевые аргументы
1 — вердикт верный, но обоснование неполное или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение неверно. F1-score — это гармоническое среднее между precision и recall. Если F1-
score увеличился, это не обязательно означает, что и precision, и recall увеличились. Например,
возможно, что precision уменьшилось, а recall увеличилось в большей степени, что привело к
росту F1-score. Таким образом, рост F1-score не гарантирует одновременного роста и precision, и
recall.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B018 · тип B

Задача: Верно или неверно утверждение: «Увеличение числа соседей k в kNN всегда уменьшает ошибку на тестовой выборке, потому что модель меньше переобучается»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
- рост k уменьшает дисперсию (граница сглаживается), но увеличивает смещение
- при слишком большом k модель недообучается и тестовая ошибка растёт; при k = n — всегда самый
частый класс
- зависимость тестовой ошибки от k обычно U-образная, а не монотонная

Типичные ошибки:
- ответ «верно» со ссылкой на уменьшение переобучения
- путать ошибку на обучающей и на тестовой выборке

2 — «неверно» и объяснено, что большой k ведёт к недообучению (рост смещения)
1 — «неверно», но обоснование неполное (нет недообучения/смещения) или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение неверно. Увеличение числа соседей $k$ в алгоритме k-ближайших соседей (kNN)
действительно снижает риск переобучения модели, поскольку усреднение большего количества точек
приводит к менее чувствительной зависимости от отдельных наблюдений. Однако чрезмерное
увеличение $k$ может привести к другой проблеме — недостаточной адаптации модели к данным
(underfitting). Это происходит потому, что при большом значении $k$ учитываются точки, которые
находятся далеко от рассматриваемой точки, что размывает границы классов и ухудшает точность
классификации или регрессии.

Таким образом, существует оптимальное значение $k$, которое обеспечивает баланс между
переобучением и недообучением. Выбор слишком большого значения $k$ может увеличить ошибку на
тестовой выборке вместо уменьшения.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B019 · тип B

Задача: Верно или неверно утверждение: «PCA выбирает направления, лучше всего разделяющие классы»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
– PCA — метод без учителя: метки классов не используются
– PCA ищет направления максимальной дисперсии; они могут не разделять классы
– направления, разделяющие классы, ищет LDA (линейный дискриминант Фишера)

Типичные ошибки:
– ответ «верно», путая PCA и LDA
– считать, что большая дисперсия всегда означает лучшее разделение

2 — «неверно» + хотя бы один из доводов: PCA не использует метки или максимизирует дисперсию, а не разделимость
1 — «неверно», но обоснование с ошибкой или без этих доводов
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение неверно. PCA (Principal Component Analysis) является методом снижения размерности, который ищет направления наибольшей дисперсии в данных, а не направления, наилучшим образом разделяющие классы. Основная цель PCA — найти ортогональные компоненты, которые объясняют большую часть вариации в данных, независимо от меток классов. Методы, направленные на разделение классов, включают LDA (Linear Discriminant Analysis), который специально оптимизирован для разделения классов.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B020 · тип B

Задача: Верно или неверно утверждение: «Алгоритм кластеризации с более высоким средним коэффициентом силуэта может хуже восстанавливать истинную структуру групп в данных, чем алгоритм с более низким коэффициентом силуэта»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– коэффициент силуэта — внутренняя метрика: оценивает компактность кластеров и их отделённость друг от друга, истинные метки не использует
– высокий силуэт не гарантирует соответствия кластеров истинным группам объектов
– метрика предпочитает компактные (выпуклые) кластеры, даже если реальные группы имеют сложную геометрическую форму
– пример: k-means может разрезать кольцеобразный кластер на части и получить силуэт выше, чем HDBSCAN, который корректно выделит исходную структуру
```

```
Типичные ошибки:
– утверждение, что максимальный коэффициент силуэта всегда соответствует наилучшей
кластеризации
– путаница между внутренними и внешними метриками качества кластеризации
– предположение, что коэффициент силуэта непосредственно измеряет соответствие истинным меткам
– игнорирование влияния формы кластеров на оценку качества

2 — правильно определена истинность утверждения («верно»), объяснено ограничение коэффициента
силуэта и приведён пример
1 — правильно определена истинность, но обоснование неполное
0 — неверный ответ или некорректное обоснование
```

Ответ модели:

```
Коэффициент силуэта — это метрика, которая оценивает качество кластеризации. Он учитывает как
расстояние между точками внутри одного кластера (внутренние расстояния), так и расстояние между
разными кластерами (внешние расстояния). Чем выше коэффициент силуэта, тем лучше алгоритм
разделяет данные на кластеры, и тем ближе полученные кластеры к истинной структуре данных.

Следовательно, утверждение «Алгоритм кластеризации с более высоким средним коэффициентом
силуэта может хуже восстанавливать истинную структуру групп в данных, чем алгоритм с более
низким коэффициентом силуэта» **неверно**. Высокий коэффициент силуэта указывает на более
качественную кластеризацию и, соответственно, на лучшее восстановление истинной структуры
данных.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B021 · тип B

Задача: Верно или неверно утверждение: «Увеличение числа соседей k в kNN всегда уменьшает ошибку на тестовой выборке, потому что модель меньше переобучается»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
– рост k уменьшает дисперсию (граница сглаживается), но увеличивает смещение
– при слишком большом k модель недообучается и тестовая ошибка растёт; при k = n — всегда самый
частый класс
– зависимость тестовой ошибки от k обычно U-образная, а не монотонная

Типичные ошибки:
– ответ «верно» со ссылкой на уменьшение переобучения
– путать ошибку на обучающей и на тестовой выборке

2 — «неверно» и объяснено, что большой k ведёт к недообучению (рост смещения)
1 — «неверно», но обоснование неполное (нет недообучения/смещения) или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение неверно. Увеличение числа соседей $ k $ в методе $ k $-ближайших соседей ($ k $-NN)
не всегда уменьшает ошибку на тестовой выборке. При увеличении $ k $ модель становится более
обобщенной, так как учитывает больше соседей, но это может привести к ухудшению
производительности, если $ k $ становится слишком большим. В этом случае модель может
```

```
недообучаться, теряя способность улавливать сложные паттерны в данных. Таким образом,
увеличение $ k $ не гарантирует уменьшения ошибки на тестовой выборке.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B022 · тип B

Задача: Верно или неверно утверждение: «Алгоритм кластеризации с более высоким средним коэффициентом силуэта может хуже восстанавливать истинную структуру групп в данных, чем алгоритм с более низким коэффициентом силуэта»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– коэффициент силуэта – внутренняя метрика: оценивает компактность кластеров и их отделённость
друг от друга, истинные метки не использует
– высокий силуэт не гарантирует соответствия кластеров истинным группам объектов
– метрика предпочитает компактные (выпуклые) кластеры, даже если реальные группы имеют сложную
геометрическую форму
– пример: k-means может разрезать кольцеобразный кластер на части и получить силуэт выше, чем
HDBSCAN, который корректно выделит исходную структуру

Типичные ошибки:
– утверждение, что максимальный коэффициент силуэта всегда соответствует наилучшей
кластеризации
– путаница между внутренними и внешними метриками качества кластеризации
– предположение, что коэффициент силуэта непосредственно измеряет соответствие истинным меткам
– игнорирование влияния формы кластеров на оценку качества

2 – правильно определена истинность утверждения («верно»), объяснено ограничение коэффициента
силуэта и приведён пример
1 – правильно определена истинность, но обоснование неполное
0 – неверный ответ или некорректное обоснование
```

Ответ модели:

```
Утверждение верно. Коэффициент силуэта измеряет степень разделения и плотности точек внутри
кластеров, однако он не всегда отражает истинную структуру данных. Например, если истинная
структура данных сильно отличается от предположений модели (например, наличие сложных
нелинейных границ между кластерами), то даже высокая величина коэффициента силуэта не
гарантирует правильного восстановления структуры. Таким образом, алгоритм с более низким
коэффициентом силуэта может лучше отражать реальную структуру данных, если эта структура
сложнее, чем предполагает метрика силуэта.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 90
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B023 · тип B

Задача: Верно или неверно утверждение: «Если добавить новый признак в линейную регрессию без регуляризации и заново обучить модель методом МНК, ошибка на обучающих данных не увеличится, даже если новый признак бесполезный»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– МНК подбирает коэффициенты так, чтобы минимизировать сумму квадратов ошибок
– если новый признак не помогает, модель может просто поставить его коэффициент равным нулю
– поэтому минимальная ошибка MSE на обучающих данных не может увеличиться, но на тестовых
данных качество может ухудшиться

Типичные ошибки:
– утверждение, что бесполезный признак обязательно увеличит ошибку на обучении
– путаница между ошибкой на обучающих и тестовых данных
– утверждение, что добавление признака всегда улучшает качество модели

2 – «верно» и объяснено, что старое решение с нулевым коэффициентом при новом признаке остаётся
допустимым, поэтому минимум ошибки на обучении не растёт
1 – «верно», но обоснование неполное (нет аргумента про нулевой коэффициент / вложенность
моделей) или с ошибкой
0 – вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение верно. Добавление нового признака в линейную регрессию без регуляризации всегда
приводит к тому, что метод наименьших квадратов (МНК) найдет такое значение коэффициента для
нового признака, которое минимизирует ошибку на обучающем множестве. Даже если новый признак
бесполезный, коэффициент при нем может стать равным нулю или близкому к нему значению, что не
ухудшит качество модели на обучении. Таким образом, ошибка на обучающей выборке не увеличится.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B024 · тип B

Задача: Верно или неверно утверждение: «Увеличение числа соседей k в kNN всегда уменьшает ошибку на тестовой выборке, потому что модель меньше переобучается»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
– рост k уменьшает дисперсию (граница сглаживается), но увеличивает смещение
– при слишком большом k модель недообучается и тестовая ошибка растёт; при k = n – всегда самый
частый класс
– зависимость тестовой ошибки от k обычно U-образная, а не монотонная

Типичные ошибки:
– ответ «верно» со ссылкой на уменьшение переобучения
– путать ошибку на обучающей и на тестовой выборке

2 – «неверно» и объяснено, что большой k ведёт к недообучению (рост смещения)
```

```
1 — «неверно», но обоснование неполное (нет недообучения/смещения) или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение неверно. Увеличение числа соседей \( k \) в алгоритме kNN не всегда уменьшает
ошибку на тестовой выборке и не всегда предотвращает переобучение. При слишком большом \( k \)
модель может стать недостаточно чувствительной к локальным особенностям данных и начать
сглаживать важные паттерны, что приведёт к увеличению ошибки из-за недообучения. С другой
стороны, при малом \( k \) модель может переобучиться, слишком точно следуя шуму в данных.
Таким образом, оптимальное значение \( k \) зависит от конкретного набора данных и требует
подбора на основе анализа валидационной выборки.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B025 · тип B

Задача: Верно или неверно утверждение: «Если у модели вырос F1-score, значит и precision, и recall тоже обязательно стали выше»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
- F1-score — это гармоническое среднее precision и recall
- F1 может увеличиться, даже если одна из двух метрик уменьшилась, а другая выросла достаточно
сильно
- например, при precision = 0.8 и recall = 0.4 значение F1 примерно равно 0.53, а при precision
= 0.7 и recall = 0.6 оно увеличится до 0.65, хотя precision снизился

Типичные ошибки:
- утверждение, что рост F1 всегда означает рост обеих метрик
- путаница между F1-score, precision и recall
- непонимание того, что увеличение одной метрики может компенсировать снижение другой

2 — вердикт верный и обоснование содержит ключевые аргументы
1 — вердикт верный, но обоснование неполное или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение неверно. F1-score является гармоническим средним между точностью (precision) и
полнотой (recall):

$$F_1 = \frac{2 \cdot precision \cdot recall}{precision + recall}$$

Это означает, что даже если один из показателей (precision или recall) снизился, другой
показатель может увеличиться настолько сильно, что итоговый F1-score вырастет. Например,
значительное увеличение recall может компенсировать небольшое снижение precision, приводя к
росту общего F1-score.

Таким образом, рост F1-score не гарантирует одновременное повышение обоих метрик precision и
recall.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B026 · тип B

Задача: Верно или неверно утверждение: «Случайный лес не требует масштабирования признаков, а SVM с RBF-ядром требует»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
- деревья разбивают по порогу одного признака: монотонное преобразование признака не меняет
разбиения
- RBF-ядро зависит от евклидова расстояния: признак с большим масштабом доминирует, gamma
теряет смысл

Типичные ошибки:
- ответ «неверно», потому что «масштабирование нужно всегда»
- утверждать, что случайному лесу масштабирование заметно помогает

2 — «верно» и названы обе причины (пороги в деревьях, расстояния в RBF)
1 — «верно», но объяснена только одна часть или есть ошибка в объяснении
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение **верно**.

**Обоснование:**

- **Случайный лес** строит решающие деревья, которые принимают решения на основе сравнений вида
«признак больше или меньше порога». Такие сравнения инвариантны к монотонным преобразованиям
признаков, в том числе к масштабированию. Поэтому случайный лес не требует нормализации или
стандартизации признаков.

- **SVM с RBF-ядром** использует евклидово расстояние между точками:
  \( K(x, x') = \exp(-\gamma \|x - x'\|^2) \).
  Если признаки имеют разные масштабы, признаки с большим диапазоном значений будут
доминировать в расстоянии, искажая форму ядра. Поэтому для SVM с RBF-ядром масштабирование
признаков важно.

Таким образом, утверждение корректно.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 99
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B027 · тип B

Задача: Верно или неверно утверждение: «Если в логистической регрессии увеличить порог с 0.5 до 0.7, вероятности, которые выдаёт модель, не изменятся, но объектов, отнесённых к положительному классу, может стать меньше»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– логистическая регрессия сначала вычисляет вероятность принадлежности объекта к положительному
классу
– порог используется только для перевода вероятности в метку класса, поэтому его изменение не
влияет на сами вероятности
– при увеличении порога часть объектов, которые раньше относились к положительному классу,
может перейти в отрицательный

Типичные ошибки:
– утверждение, что изменение порога меняет вероятности, рассчитанные моделью
– путаница между вероятностью и предсказанной меткой класса
– утверждение, что увеличение порога обязательно изменит количество положительных предсказаний

2 — вердикт верный и обоснование содержит ключевые аргументы
1 — вердикт верный, но обоснование неполное или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Логистическая регрессия выдаёт вероятности принадлежности объекта к положительному классу.
Изменение порога классификации с 0,5 до 0,7 не влияет на сами вероятности, которые модель
предсказывает — они остаются такими же. Однако при повышении порога количество объектов,
которые будут отнесены к положительному классу, действительно может уменьшиться, так как для
отнесения к положительному классу теперь требуется более высокая вероятность.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B028 · тип B

Задача: Верно или неверно утверждение: «Если у модели вырос F1-score, значит и precision, и recall тоже обязательно стали выше»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
– F1-score — это гармоническое среднее precision и recall
– F1 может увеличиться, даже если одна из двух метрик уменьшилась, а другая выросла достаточно
сильно
– например, при precision = 0.8 и recall = 0.4 значение F1 примерно равно 0.53, а при precision
= 0.7 и recall = 0.6 оно увеличится до 0.65, хотя precision снизился

Типичные ошибки:
– утверждение, что рост F1 всегда означает рост обеих метрик
– путаница между F1-score, precision и recall
– непонимание того, что увеличение одной метрики может компенсировать снижение другой

2 — вердикт верный и обоснование содержит ключевые аргументы
1 — вердикт верный, но обоснование неполное или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение неверно, так как F1-score — это гармоническое среднее между precision и recall, и
его рост не обязательно означает одновременный рост обоих показателей. Например, если precision
снизилась, а recall значительно увеличился, F1-score всё равно может вырасти. Таким образом,
изменение одного показателя может компенсировать изменение другого, и общий F1-score может
увеличиться, даже если один из компонентов (precision или recall) уменьшился.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B029 · тип B

Задача: Верно или неверно утверждение: «Наивный байесовский классификатор даёт хорошо откалиброванные вероятности, потому что он вероятностный»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
- предположение об условной независимости признаков обычно нарушено: зависимые признаки
учитываются многократно, вероятности смещаются к 0 и 1 (переуверенность)
- вероятностная модель не означает калиброванных вероятностей; калибровку проверяют диаграммой
надёжности
- исправляется калибровкой (Platt, isotonic, CalibratedClassifierCV)

Типичные ошибки:
- ответ «верно», раз модель «выдаёт вероятности»
- путать хорошую точность классификации с хорошей калибровкой

2 — «неверно» и названа причина (нарушение независимости → переуверенные вероятности)
1 — «неверно», но причина не названа или объяснена с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение **неверно**.

Наивный байесовский классификатор действительно основан на вероятностной модели и выдаёт
значения, которые формально можно интерпретировать как вероятности принадлежности классам.
Однако на практике эти вероятности часто оказываются плохо откалиброванными.

Основные причины:

1. **Сильное предположение о независимости признаков.**
   Наивный байесовский классификатор предполагает, что признаки условно независимы при заданном
классе. В реальных данных это предположение часто нарушается, из-за чего итоговые вероятности
могут быть чрезмерно «уверенными» — близкими к 0 или 1, даже когда истинная неопределённость
выше.

2. **Некорректная спецификация модели.**
   Если распределения признаков выбраны неверно (например, используется гауссовское
распределение для заведомо негауссовских данных), вероятности могут систематически отклоняться
от истинных.

3. **Смещение оценок.**
   Наивный байесовский классификатор оптимизирует не калибровку, а точность классификации.
Поэтому даже будучи «вероятностным» по построению, он не гарантирует, что, например, среди
объектов с предсказанной вероятностью 0.8 действительно 80% принадлежат положительному классу.
```

```
Таким образом, вероятностная природа модели сама по себе не обеспечивает хорошую калибровку.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B030 · тип B

Задача: Верно или неверно утверждение: «Если добавить новый признак в линейную регрессию без регуляризации и заново обучить модель методом МНК, ошибка на обучающих данных не увеличится, даже если новый признак бесполезный»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– МНК подбирает коэффициенты так, чтобы минимизировать сумму квадратов ошибок
– если новый признак не помогает, модель может просто поставить его коэффициент равным нулю
– поэтому минимальная ошибка MSE на обучающих данных не может увеличиться, но на тестовых
данных качество может ухудшиться

Типичные ошибки:
– утверждение, что бесполезный признак обязательно увеличит ошибку на обучении
– путаница между ошибкой на обучающих и тестовых данных
– утверждение, что добавление признака всегда улучшает качество модели

2 — «верно» и объяснено, что старое решение с нулевым коэффициентом при новом признаке остаётся
допустимым, поэтому минимум ошибки на обучении не растёт
1 — «верно», но обоснование неполное (нет аргумента про нулевой коэффициент / вложенность
моделей) или с ошибкой
0 — вердикт неверный или обоснования нет
```

Ответ модели:

```
Обоснование: утверждение верно, потому что метод наименьших квадратов (МНК) стремится
минимизировать сумму квадратов отклонений предсказаний модели от фактических значений. При
добавлении нового признака модель получает дополнительную степень свободы для минимизации этой
суммы. Даже если новый признак бесполезен, модель может просто «игнорировать» его, установив
соответствующий коэффициент близким к нулю, но при этом ошибка на обучающих данных не
увеличится.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B031 · тип B

Задача: Верно или неверно утверждение: «Увеличение числа соседей k в kNN всегда уменьшает ошибку на тестовой выборке, потому что модель меньше переобучается»? Обоснуйте ответ.

> **Эталон:** утверждение неверно

Рубрика / чек-лист:

```
Ключевые аргументы:
– рост k уменьшает дисперсию (граница сглаживается), но увеличивает смещение
– при слишком большом k модель недообучается и тестовая ошибка растёт; при k = n – всегда самый частый класс
– зависимость тестовой ошибки от k обычно U-образная, а не монотонная

Типичные ошибки:
– ответ «верно» со ссылкой на уменьшение переобучения
– путать ошибку на обучающей и на тестовой выборке

2 – «неверно» и объяснено, что большой k ведёт к недообучению (рост смещения)
1 – «неверно», но обоснование неполное (нет недообучения/смещения) или с ошибкой
0 – вердикт неверный или обоснования нет
```

Ответ модели:

```
Утверждение **неверно**.

Увеличение числа соседей \( k \) в методе k ближайших соседей (kNN) действительно обычно снижает переобучение и делает границу решений более гладкой. Однако это **не всегда** уменьшает ошибку на тестовой выборке.

При слишком большом \( k \) модель может стать чрезмерно смещённой (недообученной), потому что она начинает учитывать слишком много далёких точек, в том числе из других классов. Это может привести к росту ошибки на тесте. Оптимальное \( k \) обычно подбирается по кросс-валидации, и зависимость ошибки от \( k \) имеет U-образный характер: сначала ошибка может уменьшаться, затем снова расти.

Таким образом, утверждение о том, что увеличение \( k \) **всегда** уменьшает ошибку на тестовой выборке, неверно.

ОТВЕТ: неверно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

B032 · тип B

Задача: Верно или неверно утверждение: «Случайный лес не требует масштабирования признаков, а SVM с RBF-ядром требует»? Обоснуйте ответ.

> **Эталон:** утверждение верно

Рубрика / чек-лист:

```
Ключевые аргументы:
– деревья разбивают по порогу одного признака: монотонное преобразование признака не меняет разбиения
– RBF-ядро зависит от евклидова расстояния: признак с большим масштабом доминирует, gamma теряет смысл

Типичные ошибки:
– ответ «неверно», потому что «масштабирование нужно всегда»
– утверждать, что случайному лесу масштабирование заметно помогает

2 – «верно» и названы обе причины (пороги в деревьях, расстояния в RBF)
1 – «верно», но объяснена только одна часть или есть ошибка в объяснении
0 – вердикт неверный или обоснования нет
```

Ответ модели:

```
Случайный лес (Random Forest) не чувствителен к масштабированию признаков, так как метод
основан на деревьях решений, которые разбивают пространство признаков независимо от масштаба
значений. В то же время SVM с RBF-ядром чувствителен к масштабу признаков, поскольку функция
ядра RBF зависит от евклидова расстояния между точками, и разные масштабы признаков могут
исказить эти расстояния, влияя на качество модели.

ОТВЕТ: верно
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C001 · тип C

Задача: В сервис поддержки поступают короткие текстовые заявки. Есть 300 размеченных заявок, которые относятся к 3 категориям: проблемы с оборудованием, сетью или ПО. В качестве признаков используются частоты слов в тексте, всего около 1500 признаков. Нужно автоматически определять категорию новых заявок. Каждую ночь модель должна дообучаться на новых заявках инкрементально — без хранения старых данных и без итеративной оптимизации. При этом модель должна быстро работать, потреблять мало памяти, а её решения должно быть легко объяснить. Какой метод выбрать и почему?

> **Эталон:** Наивный Байес (MultinomialNB) со сглаживанием Лапласа, так как он подходит для классификации текстов по частотам слов, хорошо работает на небольших выборках, быстро обучается и позволяет интерпретировать влияние слов на предсказание.

Рубрика / чек-лист:

```
Приемлемые методы:
- MultinomialNB со сглаживанием Лапласа
- BernoulliNB, если вместо частот слов использовать признаки их наличия или отсутствия
- ComplementNB (вариант наивного Байеса, устойчивый к несбалансированным классам)

Обязательные аргументы (все нужны для 2):
- размеченных данных мало, а признаков много, поэтому нужен метод, который может работать на
небольших выборках без сложного обучения
- MultinomialNB подходит для текстов, представленных частотами слов
- наивный Байес быстро обучается, требует относительно мало памяти и позволяет объяснять
предсказания через вероятности слов для разных классов
- наивный Байес дообучается инкрементально простым обновлением счётчиков слов по классам
(partial_fit), без хранения старых данных и без итеративной оптимизации

Типичные ошибки:
- выбор сложной нейросети без учёта малого количества размеченных данных и ограничений по
ресурсам
- выбор GaussianNB без объяснения, почему он подходит для частот слов
- утверждение, что наивный Байес требует большого количества данных для обучения
- отсутствие объяснения, почему выбранный метод подходит для текстовых признаков

2 — выбран подходящий вариант наивного Байеса (MultinomialNB, BernoulliNB, ComplementNB) и
приведены все обязательные аргументы
1 — выбран наивный Байес, но обоснование неполное или пропущены некоторые аргументы; ЛИБО
выбрана логистическая регрессия (в т. ч. SGDClassifier с partial_fit) с верными доводами, но
без учёта требования дообучаться без итеративной оптимизации
0 — выбран неподходящий метод (например, сложная нейросеть или kNN без оговорок про малый объём
и ресурсы) или отсутствует корректное обоснование
```

Ответ модели:

```
Для данной задачи подходит метод **логистическая регрессия**.

Обоснование:
1. **Малая память и скорость**: Логистическая регрессия имеет простую структуру и может
обучаться быстро, что подходит для задач с ограниченной памятью и высокой производительностью.
2. **Инкрементальное обучение**: Метод поддерживает инкрементное обучение (online learning),
что позволяет модели дообучаться на новых данных без хранения старых.
3. **Объяснимость**: Логистическая регрессия предоставляет веса для каждого признака, что
позволяет легко интерпретировать решения модели.
4. **Малое количество признаков**: Несмотря на 1500 признаков, логистическая регрессия может
справиться с ними, особенно если использовать регуляризацию (например, L2).

ОТВЕТ: Логистическая регрессия
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C002 · тип C

Задача: На промышленном предприятии система мониторинга собирает данные с 50 датчиков, установленных на производственном оборудовании. Датчики измеряют температуру, давление, вибрацию и энергопотребление. Накоплено 120 000 наблюдений, каждое описывается 50 числовыми признаками с разными единицами измерения. Предварительный анализ показал сильную корреляцию между многими признаками. Требуется уменьшить размерность данных для сокращения объёма хранения и ускорения последующего анализа, сохранив не менее 95% общей дисперсии. Меток классов нет. Физическая интерпретируемость каждого нового признака не обязательна, но важно минимизировать потери информации. Преобразование новых наблюдений должно выполняться быстро, а хранение преобразованных данных должно требовать меньше памяти. Какой метод выбрать и почему?

> **Эталон:** Метод главных компонент (PCA) после стандартизации числовых признаков. PCA строит ортогональные линейные комбинации исходных признаков, максимизирующие сохраняемую дисперсию. При сильной корреляции позволяет сократить размерность за счёт избыточной информации. Число компонент выбирается по накопленной доле объяснённой дисперсии не менее 95%. Стандартизация необходима из-за разных масштабов измерений. PCA не требует меток классов. Преобразование обучается на обучающей выборке и затем применяется к новым данным. Ограничения: ухудшение интерпретируемости признаков и потеря части информации; сохранение дисперсии не гарантирует сохранения всей информации, полезной для последующих задач.

Рубрика / чек-лист:

```
Приемлемые методы:
- PCA
- Principal Component Analysis
- Метод главных компонент
- StandardScaler + PCA
- IncrementalPCA при обоснованных ограничениях памяти
- TruncatedSVD на стандартизированных (центрированных) данных — математически совпадает с PCA

Обязательные аргументы (все нужны для 2):
- выбран PCA как метод снижения размерности числовых данных без использования меток классов
- объяснено, что сильная корреляция признаков позволяет сократить размерность за счёт
избыточности информации
- указана необходимость стандартизации признаков перед PCA из-за различных единиц измерения и
масштабов
```

```
– предложен выбор минимального числа компонент, обеспечивающего накопленную объяснённую
дисперсию не менее 95%

Дополнительно (не обязательно):
– объяснено, что уменьшение числа признаков сокращает объём хранения преобразованных данных и
может ускорить последующую обработку
– отмечено ограничение PCA: главные компоненты являются линейными комбинациями исходных
признаков и хуже поддаются физической интерпретации
– указано, что параметры стандартизации и PCA обучаются только на обучающих данных, а к новым
наблюдениям применяется уже построенное преобразование

Типичные ошибки:
– предложить обучение с учителем, несмотря на отсутствие меток классов
– применить PCA без стандартизации и не учесть различие масштабов признаков
– утверждать, что PCA просто выбирает несколько исходных признаков
– считать, что PCA гарантирует сохранение 95% информации при заранее фиксированном числе
компонент
– утверждать, что PCA всегда улучшает качество последующей модели
– игнорировать потерю интерпретируемости главных компонент
– обучать PCA на всей выборке до разделения на обучающую и тестовую при последующей оценке
модели

2 – выбран PCA, обоснованный вариант IncrementalPCA или TruncatedSVD на стандартизированных
данных и раскрыты все четыре обязательных аргумента (без меток; корреляция → избыточность;
стандартизация из-за разных единиц; число компонент по накопленной дисперсии ≥ 95%);
дополнительные аргументы желательны, но не обязательны
1 – выбран PCA, основная идея снижения размерности объяснена правильно, но раскрыты не все
обязательные аргументы либо есть отдельные неточности, не меняющие вывод
0 – предложено обучение с учителем, PCA описан как отбор исходных признаков или как
классификатор, выбран неподходящий метод или нет обоснования
```

Ответ модели:

```
В данной ситуации наиболее подходящим методом уменьшения размерности данных является метод
главных компонент (PCA – Principal Component Analysis). Он позволяет сократить количество
признаков, сохранив при этом максимально возможное количество информации (дисперсии) в данных.
PCA хорошо подходит для данных с сильной корреляцией между признаками, так как выделяет новые
ортогональные признаки (главные компоненты), которые максимально объясняют вариативность
исходных данных. При этом метод не требует наличия меток классов, что соответствует условиям
задачи. Преобразование новых наблюдений с помощью PCA выполняется быстро, а хранение данных
после уменьшения размерности требует меньше памяти, что также отвечает поставленным
требованиям.

ОТВЕТ: метод главных компонент (PCA)
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C003 · тип C

Задача: В сервис поддержки поступают короткие текстовые заявки. Есть 300 размеченных заявок, которые относятся к 3 категориям: проблемы с оборудованием, сетью или ПО. В качестве признаков используются частоты слов в тексте, всего около 1500 признаков. Нужно автоматически определять категорию новых заявок. Каждую ночь модель должна дообучаться на новых заявках инкрементально — без хранения старых данных и без итеративной оптимизации. При этом модель должна быстро работать, потреблять мало памяти, а её решения должно быть легко объяснить. Какой метод выбрать и почему?

> **Эталон:** Наивный Байес (MultinomialNB) со сглаживанием Лапласа, так как он подходит для классификации текстов по частотам слов, хорошо работает на небольших выборках, быстро обучается и позволяет интерпретировать влияние слов на предсказание.

Рубрика / чек-лист:

```
Приемлемые методы:
– MultinomialNB со сглаживанием Лапласа
– BernoulliNB, если вместо частот слов использовать признаки их наличия или отсутствия
– ComplementNB (вариант наивного Байеса, устойчивый к несбалансированным классам)

Обязательные аргументы (все нужны для 2):
– размеченных данных мало, а признаков много, поэтому нужен метод, который может работать на небольших выборках без сложного обучения
– MultinomialNB подходит для текстов, представленных частотами слов
– наивный Байес быстро обучается, требует относительно мало памяти и позволяет объяснять предсказания через вероятности слов для разных классов
– наивный Байес дообучается инкрементально простым обновлением счётчиков слов по классам (partial_fit), без хранения старых данных и без итеративной оптимизации

Типичные ошибки:
– выбор сложной нейросети без учёта малого количества размеченных данных и ограничений по ресурсам
– выбор GaussianNB без объяснения, почему он подходит для частот слов
– утверждение, что наивный Байес требует большого количества данных для обучения
– отсутствие объяснения, почему выбранный метод подходит для текстовых признаков

2 – выбран подходящий вариант наивного Байеса (MultinomialNB, BernoulliNB, ComplementNB) и приведены все обязательные аргументы
1 – выбран наивный Байес, но обоснование неполное или пропущены некоторые аргументы; ЛИБО выбрана логистическая регрессия (в т. ч. SGDClassifier с partial_fit) с верными доводами, но без учёта требования дообучаться без итеративной оптимизации
0 – выбран неподходящий метод (например, сложная нейросеть или kNN без оговорок про малый объём и ресурсы) или отсутствует корректное обоснование
```

Ответ модели:

```
Для решения задачи классификации текстовых заявок с учётом ограничений (инкрементальное обучение без хранения старых данных, низкая потребность в памяти, высокая скорость работы и объяснимость моделей) лучше всего подойдёт метод наивного байесовского классификатора.

**Обоснование выбора:**
1. **Инкрементальное обучение:** наивный байесовский классификатор позволяет обновлять модель на новых данных без необходимости хранить весь набор исторических данных. Для обновления достаточно пересчитывать частоты слов в рамках каждой категории.
2. **Низкие требования к памяти:** модель хранит только вероятности (априорные и условные), что делает её очень лёгкой с точки зрения потребления памяти.
3. **Скорость работы:** вычисления с наивным байесовским классификатором выполняются быстро, что важно для обработки поступающих заявок.
4. **Объяснимость:** результаты классификации легко интерпретировать, так как можно увидеть, какие слова и с какой вероятностью влияют на отнесение заявки к определённой категории.
5. **Подход к обработке текста:** использование частот слов в качестве признаков хорошо сочетается с наивным байесовским подходом, который часто применяется в задачах текстовой классификации.

ОТВЕТ: Наивный байесовский классификатор
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C004 · тип C

Задача: Нужно предсказывать брак партии на производственной линии. Есть 50 000 партий, 30 числовых параметров процесса и 10 категориальных (поставщик сырья, смена, линия), в части признаков пропуски, зависимости нелинейные, брак — 8% партий. Важна максимальная точность ранжирования (ROC AUC), интерпретируемость не обязательна, обучение на одном ноутбуке. Какой метод выбрать и почему?

> **Эталон:** градиентный бустинг над деревьями (CatBoost / LightGBM / HistGradientBoosting)

Рубрика / чек-лист:

```
Приемлемые методы:
- градиентный бустинг над деревьями: CatBoost, LightGBM, XGBoost,
HistGradientBoostingClassifier
- случайный лес — допустим с обоснованием (обычно уступает бустингу на таких данных)

Обязательные аргументы (все нужны для 2):
- табличные данные смешанного типа с нелинейностями — сильная сторона ансамблей деревьев
- пропуски и категориальные признаки: встроенная обработка (HistGB, CatBoost, LightGBM) или
явное кодирование
- дисбаланс 8%: метрика ROC AUC или PR AUC, стратифицированная кросс-валидация, при
необходимости веса классов

Дополнительно (не обязательно):
- подбор гиперпараметров (learning rate, глубина, число деревьев / early stopping) на кросс-
валидации

Типичные ошибки:
- глубокая нейросеть без обоснования для табличных данных такого размера
- kNN или SVM с RBF-ядром без обработки пропусков и категорий
- оценка по accuracy при 8% брака
- one-hot кодирование категорий с большим числом значений без оговорок

2 — бустинг (или лес с обоснованием) + все три обязательных аргумента (табличные смешанные
данные, пропуски/категории, дисбаланс)
1 — приемлемый метод, но пропущен обязательный аргумент; либо обоснование с ошибкой
0 — неприемлемый метод или нет обоснования
```

Ответ модели:

```
Для задачи предсказания брака партии на производственной линии с учётом нелинейных
зависимостей, наличия пропусков в данных и требования максимальной точности ранжирования (ROC
AUC) наиболее подходящим методом будет градиентный бустинг. Этот метод хорошо справляется с
нелинейными зависимостями и может эффективно работать с комбинацией числовых и категориальных
признаков. Он также устойчив к пропускам в данных при правильном предобработке и способен
достигать высокой точности предсказаний, что важно для задачи с относительно низким процентом
брака (8%). Градиентный бустинг можно реализовать и обучить на одном ноутбуке, что
соответствует ограничениям задачи.

ОТВЕТ: градиентный бустинг
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C005 · тип C

Задача: На производстве нужно определять, исправно ли оборудование, по показаниям датчиков. Есть 2000 размеченных записей и 20 числовых признаков: температура, давление,

вибрация и другие показатели. Из анализа данных видно, что классы нельзя хорошо разделить прямой линией. Модель должна быстро работать на обычном процессоре, не требовать много памяти. По регламенту сертификации допускается только одна модель — ансамбли (случайный лес, бустинг, стекинг) использовать нельзя. Подробно объяснять каждое предсказание не обязательно. Какой метод выбрать и почему?

> **Эталон:** SVM с RBF-ядром. Он позволяет строить нелинейную границу между классами и подходит для выборки такого размера. Признаки нужно масштабировать, а параметры C и gamma подобрать с помощью кросс-валидации.

Рубрика / чек-лист:

```
Приемлемые методы:
– SVM с RBF–ядром
– SVM с полиномиальным ядром при обоснованном выборе параметров

Обязательные аргументы (все нужны для 2):
– классы разделяются нелинейно, поэтому обычного линейного классификатора может быть
недостаточно
– SVM с нелинейным ядром позволяет построить более сложную границу между классами, не добавляя
новые признаки вручную
– размер выборки позволяет использовать ядерный SVM, а для корректной работы числовые признаки
нужно масштабировать

Типичные ошибки:
– выбор линейного SVM без учёта нелинейной границы между классами
– утверждение, что SVM может строить только линейные границы
– игнорирование необходимости масштабирования признаков
– утверждение, что SVM с RBF–ядром не требует настройки параметров C и gamma
– выбор случайного леса или бустинга вопреки запрету ансамблей в условии

2 – выбран подходящий SVM с нелинейным ядром и приведены все обязательные аргументы
1 – выбран SVM с нелинейным ядром, но обоснование неполное или пропущен один из обязательных
аргументов; ЛИБО выбрана допустимая одиночная альтернатива – kNN с масштабированием или
небольшая нейросеть (MLP) – с верными доводами про нелинейную границу и ресурсы
0 – выбран неподходящий метод (линейный классификатор без учёта нелинейной границы, ансамбль
вопреки запрету регламента) или нет корректного обоснования
```

Ответ модели:

```
Для этой задачи подходит **метод опорных векторов с нелинейным ядром (SVM с RBF–ядром)**.

Обоснование:

1. **Нелинейная граница классов** – по условию данные нельзя хорошо разделить прямой линией,
значит нужен метод, способный строить нелинейные решающие правила. SVM с RBF–ядром хорошо
справляется с такими задачами.

2. **Ограничение на одну модель** – SVM является единой моделью, а не ансамблем, поэтому
соответствует регламенту сертификации.

3. **Скорость и память** – после обучения SVM с RBF–ядром предсказание выполняется быстро, так
как зависит только от числа опорных векторов. При 2000 записей и 20 признаках модель будет
компактной и быстрой на обычном процессоре.

4. **Не требуется подробная интерпретация** – SVM не даёт простого объяснения каждого
предсказания, но по условию это не обязательно.

Альтернативой могла бы быть логистическая регрессия с полиномиальными признаками, но она менее
гибкая, чем SVM с RBF, и может потребовать ручного подбора преобразований. Нейронная сеть тоже
возможна, но при таком небольшом объёме данных SVM обычно стабильнее и проще в настройке.
```

```
ОТВЕТ: SVM с RBF-ядром
УВЕРЕННОСТЬ: 92
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C006 · тип C

Задача: На промышленном предприятии система мониторинга собирает данные с 50 датчиков, установленных на производственном оборудовании. Датчики измеряют температуру, давление, вибрацию и энергопотребление. Накоплено 120 000 наблюдений, каждое описывается 50 числовыми признаками с разными единицами измерения. Предварительный анализ показал сильную корреляцию между многими признаками. Требуется уменьшить размерность данных для сокращения объёма хранения и ускорения последующего анализа, сохранив не менее 95% общей дисперсии. Меток классов нет. Физическая интерпретируемость каждого нового признака не обязательна, но важно минимизировать потери информации. Преобразование новых наблюдений должно выполняться быстро, а хранение преобразованных данных должно требовать меньше памяти. Какой метод выбрать и почему?

> **Эталон:** Метод главных компонент (PCA) после стандартизации числовых признаков. PCA строит ортогональные линейные комбинации исходных признаков, максимизирующие сохраняемую дисперсию. При сильной корреляции позволяет сократить размерность за счёт избыточной информации. Число компонент выбирается по накопленной доле объяснённой дисперсии не менее 95%. Стандартизация необходима из-за разных масштабов измерений. PCA не требует меток классов. Преобразование обучается на обучающей выборке и затем применяется к новым данным. Ограничения: ухудшение интерпретируемости признаков и потеря части информации; сохранение дисперсии не гарантирует сохранения всей информации, полезной для последующих задач.

Рубрика / чек-лист:

```
Приемлемые методы:
– PCA
– Principal Component Analysis
– Метод главных компонент
– StandardScaler + PCA
– IncrementalPCA при обоснованных ограничениях памяти
– TruncatedSVD на стандартизированных (центрированных) данных — математически совпадает с PCA

Обязательные аргументы (все нужны для 2):
– выбран PCA как метод снижения размерности числовых данных без использования меток классов
– объяснено, что сильная корреляция признаков позволяет сократить размерность за счёт
избыточности информации
– указана необходимость стандартизации признаков перед PCA из-за различных единиц измерения и
масштабов
– предложен выбор минимального числа компонент, обеспечивающего накопленную объяснённую
дисперсию не менее 95%

Дополнительно (не обязательно):
– объяснено, что уменьшение числа признаков сокращает объём хранения преобразованных данных и
может ускорить последующую обработку
– отмечено ограничение PCA: главные компоненты являются линейными комбинациями исходных
признаков и хуже поддаются физической интерпретации
– указано, что параметры стандартизации и PCA обучаются только на обучающих данных, а к новым
наблюдениям применяется уже построенное преобразование

Типичные ошибки:
```

```
– предложить обучение с учителем, несмотря на отсутствие меток классов
– применить PCA без стандартизации и не учесть различие масштабов признаков
– утверждать, что PCA просто выбирает несколько исходных признаков
– считать, что PCA гарантирует сохранение 95% информации при заранее фиксированном числе
компонент
– утверждать, что PCA всегда улучшает качество последующей модели
– игнорировать потерю интерпретируемости главных компонент
– обучать PCA на всей выборке до разделения на обучающую и тестовую при последующей оценке
модели

2 – выбран PCA, обоснованный вариант IncrementalPCA или TruncatedSVD на стандартизированных
данных и раскрыты все четыре обязательных аргумента (без меток; корреляция → избыточность;
стандартизация из-за разных единиц; число компонент по накопленной дисперсии ≥ 95%);
дополнительные аргументы желательны, но не обязательны
1 – выбран PCA, основная идея снижения размерности объяснена правильно, но раскрыты не все
обязательные аргументы либо есть отдельные неточности, не меняющие вывод
0 – предложено обучение с учителем, PCA описан как отбор исходных признаков или как
классификатор, выбран неподходящий метод или нет обоснования
```

Ответ модели:

```
Для этой задачи лучше всего подходит **метод главных компонент (PCA – Principal Component
Analysis)**.

Обоснование:

1. **Неконтролируемое обучение (нет меток классов)** – PCA не требует целевой переменной и
работает только с матрицей признаков, что соответствует условию.

2. **Сильная корреляция между признаками** – PCA специально предназначен для устранения
избыточности, вызванной корреляцией, и выделения ортогональных компонент, объясняющих
наибольшую дисперсию.

3. **Требование сохранить не менее 95% общей дисперсии** – PCA позволяет выбрать число главных
компонент, суммарная объяснённая дисперсия которых достигает заданного порога (95%). Это
напрямую управляется через параметр `n_components` или анализ explained variance ratio.

4. **Минимизация потерь информации** – среди линейных методов снижения размерности PCA
обеспечивает минимальную среднеквадратичную ошибку восстановления при фиксированном числе
компонент.

5. **Быстрое преобразование новых наблюдений** – после обучения PCA преобразование нового
вектора – это просто умножение на матрицу главных компонент, что выполняется очень быстро.

6. **Сокращение объёма хранения** – вместо 50 исходных признаков хранятся только k главных
компонент (k < 50), что уменьшает память.

7. **Физическая интерпретируемость не обязательна** – это снимает главный недостаток PCA
(компоненты являются линейными комбинациями исходных признаков и трудно интерпретируемы
физически).

Альтернативы вроде t-SNE или UMAP не подходят, так как они не сохраняют глобальную дисперсию,
не имеют быстрого линейного преобразования для новых данных и требуют больше вычислительных
ресурсов. Автоэнкодеры избыточны для этой задачи и сложнее в реализации.

ОТВЕТ: Метод главных компонент (PCA)
УВЕРЕННОСТЬ: 98
```

Балл: □ 0 □ 1 □ 2

Комментарий: _____

C007 · тип C

Задача: На производстве нужно определять, исправно ли оборудование, по показаниям датчиков. Есть 2000 размеченных записей и 20 числовых признаков: температура, давление, вибрация и другие показатели. Из анализа данных видно, что классы нельзя хорошо разделить прямой линией. Модель должна быстро работать на обычном процессоре, не требовать много памяти. По регламенту сертификации допускается только одна модель — ансамбли (случайный лес, бустинг, стекинг) использовать нельзя. Подробно объяснять каждое предсказание не обязательно. Какой метод выбрать и почему?

> **Эталон:** SVM с RBF-ядром. Он позволяет строить нелинейную границу между классами и подходит для выборки такого размера. Признаки нужно масштабировать, а параметры C и gamma подобрать с помощью кросс-валидации.

Рубрика / чек-лист:

```
Приемлемые методы:
– SVM с RBF–ядром
– SVM с полиномиальным ядром при обоснованном выборе параметров

Обязательные аргументы (все нужны для 2):
– классы разделяются нелинейно, поэтому обычного линейного классификатора может быть
недостаточно
– SVM с нелинейным ядром позволяет построить более сложную границу между классами, не добавляя
новые признаки вручную
– размер выборки позволяет использовать ядерный SVM, а для корректной работы числовые признаки
нужно масштабировать

Типичные ошибки:
– выбор линейного SVM без учёта нелинейной границы между классами
– утверждение, что SVM может строить только линейные границы
– игнорирование необходимости масштабирования признаков
– утверждение, что SVM с RBF–ядром не требует настройки параметров C и gamma
– выбор случайного леса или бустинга вопреки запрету ансамблей в условии

2 – выбран подходящий SVM с нелинейным ядром и приведены все обязательные аргументы
1 – выбран SVM с нелинейным ядром, но обоснование неполное или пропущен один из обязательных
аргументов; ЛИБО выбрана допустимая одиночная альтернатива – kNN с масштабированием или
небольшая нейросеть (MLP) – с верными доводами про нелинейную границу и ресурсы
0 – выбран неподходящий метод (линейный классификатор без учёта нелинейной границы, ансамбль
вопреки запрету регламента) или нет корректного обоснования
```

Ответ модели:

```
Для выбора модели рассмотрим ключевые требования задачи:

– Классы нельзя линейно разделить, значит нужна нелинейная модель.
– Требуется быстрая работа на обычном процессоре и минимальное использование памяти.
– Допускается только одиночная модель, ансамблевые методы запрещены.
– Объяснение каждого отдельного прогноза не требуется.

Из перечисленных требований наиболее подходящими являются алгоритмы на основе деревьев решений,
поскольку они:
1. Хорошо справляются с нелинейными задачами
2. Работают достаточно быстро даже на обычных процессорах
3. Используют относительно мало памяти
4. Не требуют ансамбля моделей

Однако случайный лес и градиентный бустинг исключаются условиями задачи. Остается одно дерево
решений.
```

```
Хотя деревья решений имеют тенденцию переобучаться на сложных задачах, в данном случае это
приемлемый компромисс между требованиями производительности и точности. Для предотвращения
переобучения можно настроить гиперпараметры дерева (ограничить глубину, минимальную частоту
классов в листьях и т.д.), чтобы получить сбалансированную модель.

ОТВЕТ: Одно дерево решений
УВЕРЕННОСТЬ: 90
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C008 · тип C

Задача: Есть 20 000 записей телеметрии насосов (6 числовых признаков: давление, расход, температура и т. п.), разметки нет. Нужно выделить типичные режимы работы. Число режимов заранее неизвестно, в данных есть выбросы (аварийные замеры), а группы на графиках вытянутые и разной плотности. Какой метод выбрать и почему?

> **Эталон:** плотностная кластеризация HDBSCAN/OPTICS после масштабирования признаков

Рубрика / чек-лист:

```
Приемлемые методы:
– HDBSCAN или OPTICS (после масштабирования признаков)
– DBSCAN – на 2 балла только с обсуждением выбора eps/min_samples при разной плотности, иначе
не выше 1
– гауссова смесь (GMM) с полными ковариациями и выбором числа компонент по BIC – с оговоркой
про выбросы

Обязательные аргументы (все нужны для 2):
– число кластеров неизвестно: метод не должен требовать k заранее или должен выбирать его по
критерию
– выбросы: плотностные методы помечают их как шум
– форма и плотность групп: k-means предполагает компактные кластеры близкого размера

Дополнительно (не обязательно):
– масштабирование признаков перед расчётом расстояний

Типичные ошибки:
– k-means с «методом локтя» без оговорок про форму кластеров и выбросы
– метод обучения с учителем (разметки нет)
– DBSCAN без обсуждения выбора eps/min_samples при разной плотности (HDBSCAN это решает)

2 – приемлемый метод + все три обязательных аргумента (неизвестное k, выбросы, форма/плотность)
1 – приемлемый метод, но пропущен обязательный аргумент; DBSCAN без обсуждения eps; k-means/GMM
с осознанными оговорками
0 – метод с учителем, k-means без оговорок или нет обоснования
```

Ответ модели:

```
В данной ситуации целесообразно использовать метод кластеризации, который позволит выделить
группы (кластеры) схожих по характеристикам записей телеметрии. Учитывая, что число режимов
заранее неизвестно, а данные содержат выбросы, подойдёт алгоритм DBSCAN (Density-Based Spatial
Clustering of Applications with Noise). Этот метод хорошо работает с данными, содержащими шумы
и выбросы, и способен выявлять кластеры произвольной формы, что важно, учитывая, что группы на
графиках вытянутые и разной плотности. DBSCAN определяет кластеры на основе плотности точек в
пространстве признаков, что позволяет эффективно отделять «нормальные» режимы работы от
аномальных замеров.
```

```
ОТВЕТ: DBSCAN
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C009 · тип C

Задача: Есть 150 размеченных записей вибросигнала, 400 спектральных признаков, 3 класса состояния подшипника. Нужно интерпретируемое решение. Какой метод выбрать и почему?

> **Эталон:** регуляризованная линейная модель (логистическая регрессия с L1/ElasticNet) или неглубокое дерево после отбора признаков

Рубрика / чек-лист:

```
Приемлемые методы:
– мультиклассовая логистическая регрессия с L1 или ElasticNet-регуляризацией
– LDA с регуляризацией ковариации (shrinkage)
– неглубокое дерево решений после отбора или агрегации признаков (например, энергии по
частотным полосам)
– линейный SVM с L1-регуляризацией и отбором признаков

Обязательные аргументы (все нужны для 2):
– признаков больше, чем объектов (400 > 150): высокий риск переобучения → регуляризация, отбор
признаков или агрегация спектра
– интерпретируемость: веса отобранных частот или правила дерева можно объяснить инженеру

Дополнительно (не обязательно):
– оценка на стратифицированной кросс-валидации: данных мало, одна отложенная выборка будет
шумной

Типичные ошибки:
– нейросеть или бустинг без оговорок про малый объём и интерпретируемость
– kNN или SVM с RBF-ядром без снижения размерности и регуляризации
– PCA как главный шаг «для интерпретируемости» (главные компоненты трудно интерпретировать)
– одно разбиение train/test на 150 объектах без кросс-валидации

2 – приемлемый метод + оба обязательных аргумента (p > n → регуляризация/отбор;
интерпретируемость)
1 – приемлемый метод, но пропущен один обязательный аргумент; либо спорный метод с верными
аргументами
0 – неприемлемый метод или нет обоснования
```

Ответ модели:

```
Для задачи с 150 наблюдениями, 400 спектральными признаками и 3 классами ключевые ограничения –
это малое число примеров относительно размерности и требование интерпретируемости.

Логистическая регрессия с L1-регуляризацией (Lasso) хорошо подходит, потому что:
– она линейна и интерпретируема: коэффициенты показывают вклад каждого спектрального признака в
каждый класс;
– L1-регуляризация автоматически отбирает небольшое число значимых признаков из 400, снижая
риск переобучения при малом числе наблюдений;
– для многоклассовой задачи используется стратегия one-vs-rest, которая остаётся понятной и
объяснимой.

Деревья решений тоже интерпретируемы, но при 400 признаках и 150 примерах склонны к
переобучению и нестабильности. Random Forest или градиентный бустинг дают высокое качество, но
их интерпретация требует дополнительных методов (SHAP, важности признаков), что усложняет
объяснение.
```

```
Поэтому оптимальный выбор — разреженная логистическая регрессия (L1), обеспечивающая баланс
между интерпретируемостью и устойчивостью при высокой размерности.

ОТВЕТ: Логистическая регрессия с L1-регуляризацией (Lasso)
УВЕРЕННОСТЬ: 92
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C010 · тип C

Задача: На производстве нужно определять, исправно ли оборудование, по показаниям датчиков. Есть 2000 размеченных записей и 20 числовых признаков: температура, давление, вибрация и другие показатели. Из анализа данных видно, что классы нельзя хорошо разделить прямой линией. Модель должна быстро работать на обычном процессоре, не требовать много памяти. По регламенту сертификации допускается только одна модель — ансамбли (случайный лес, бустинг, стекинг) использовать нельзя. Подробно объяснять каждое предсказание не обязательно. Какой метод выбрать и почему?

> **Эталон:** SVM с RBF-ядром. Он позволяет строить нелинейную границу между классами и подходит для выборки такого размера. Признаки нужно масштабировать, а параметры C и gamma подобрать с помощью кросс-валидации.

Рубрика / чек-лист:

```
Приемлемые методы:
- SVM с RBF-ядром
- SVM с полиномиальным ядром при обоснованном выборе параметров

Обязательные аргументы (все нужны для 2):
- классы разделяются нелинейно, поэтому обычного линейного классификатора может быть
недостаточно
- SVM с нелинейным ядром позволяет построить более сложную границу между классами, не добавляя
новые признаки вручную
- размер выборки позволяет использовать ядерный SVM, а для корректной работы числовые признаки
нужно масштабировать

Типичные ошибки:
- выбор линейного SVM без учёта нелинейной границы между классами
- утверждение, что SVM может строить только линейные границы
- игнорирование необходимости масштабирования признаков
- утверждение, что SVM с RBF-ядром не требует настройки параметров C и gamma
- выбор случайного леса или бустинга вопреки запрету ансамблей в условии

2 — выбран подходящий SVM с нелинейным ядром и приведены все обязательные аргументы
1 — выбран SVM с нелинейным ядром, но обоснование неполное или пропущен один из обязательных
аргументов; ЛИБО выбрана допустимая одиночная альтернатива — kNN с масштабированием или
небольшая нейросеть (MLP) — с верными доводами про нелинейную границу и ресурсы
0 — выбран неподходящий метод (линейный классификатор без учёта нелинейной границы, ансамбль
вопреки запрету регламента) или нет корректного обоснования
```

Ответ модели:

```
В условиях задачи указано, что классы нельзя хорошо разделить прямой линией, что говорит о
нелинейной природе данных. В то же время есть ограничения на использование ансамблей и
требования к быстродействию и объёму памяти. С учётом этих факторов подходящим методом может
быть метод опорных векторов (SVM) с нелинейным ядром (например, RBF). SVM способен эффективно
работать с нелинейно разделимыми данными и при этом не требует большого объёма памяти и
вычислительных ресурсов для работы на обычном процессоре после обучения.
```

```
ОТВЕТ: метод опорных векторов (SVM) с нелинейным ядром
УВЕРЕННОСТЬ: 85
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C011 · тип C

Задача: Нужно предсказывать брак партии на производственной линии. Есть 50 000 партий, 30 числовых параметров процесса и 10 категориальных (поставщик сырья, смена, линия), в части признаков пропуски, зависимости нелинейные, брак — 8% партий. Важна максимальная точность ранжирования (ROC AUC), интерпретируемость не обязательна, обучение на одном ноутбуке. Какой метод выбрать и почему?

> **Эталон:** градиентный бустинг над деревьями (CatBoost / LightGBM / HistGradientBoosting)

Рубрика / чек-лист:

```
Приемлемые методы:
– градиентный бустинг над деревьями: CatBoost, LightGBM, XGBoost,
HistGradientBoostingClassifier
– случайный лес — допустим с обоснованием (обычно уступает бустингу на таких данных)

Обязательные аргументы (все нужны для 2):
– табличные данные смешанного типа с нелинейностями — сильная сторона ансамблей деревьев
– пропуски и категориальные признаки: встроенная обработка (HistGB, CatBoost, LightGBM) или
явное кодирование
– дисбаланс 8%: метрика ROC AUC или PR AUC, стратифицированная кросс-валидация, при
необходимости веса классов

Дополнительно (не обязательно):
– подбор гиперпараметров (learning rate, глубина, число деревьев / early stopping) на кросс-
валидации

Типичные ошибки:
– глубокая нейросеть без обоснования для табличных данных такого размера
– kNN или SVM с RBF-ядром без обработки пропусков и категорий
– оценка по accuracy при 8% брака
– one-hot кодирование категорий с большим числом значений без оговорок

2 — бустинг (или лес с обоснованием) + все три обязательных аргумента (табличные смешанные
данные, пропуски/категории, дисбаланс)
1 — приемлемый метод, но пропущен обязательный аргумент; либо обоснование с ошибкой
0 — неприемлемый метод или нет обоснования
```

Ответ модели:

```
Для решения задачи прогнозирования брака партий продукции с учетом заданных условий наиболее
подходящим методом является **случайный лес** (*Random Forest*).

Обоснование выбора:

– **Нелинейность зависимостей**: Случайный лес хорошо справляется с нелинейными взаимосвязями
между признаками и целевой переменной благодаря своей структуре ансамблевого обучения на основе
деревьев решений.

– **Работа с пропущенными значениями**: Метод случайного леса устойчив к наличию пропусков в
данных и позволяет эффективно обрабатывать неполную информацию.

– **Большое количество признаков**: Алгоритм случайного леса легко масштабируется под большое
```

```
количество признаков, как числовых, так и категориальных.

- **Балансировка классов**: Несмотря на несбалансированные классы (брак составляет всего 8%),
случайный лес имеет встроенные механизмы борьбы с дисбалансом через настройку веса классов.

- **Высокая производительность на ноутбуке**: Благодаря параллельной реализации алгоритма
случайного леса, он достаточно быстро обучается даже на относительно слабых вычислительных
ресурсах ноутбука.

- **Максимальная точность ранжирования (ROC AUC)**: Случайный лес известен своей высокой
точностью в задачах бинарной классификации, особенно когда важна именно способность правильно
расставлять приоритеты (ранги) объектов.

Таким образом, учитывая указанные требования и ограничения задачи, случайный лес представляется
оптимальным выбором среди существующих методов машинного обучения.

ОТВЕТ: Random Forest
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C012 · тип C

Задача: В сервис поддержки поступают короткие текстовые заявки. Есть 300 размеченных заявок, которые относятся к 3 категориям: проблемы с оборудованием, сетью или ПО. В качестве признаков используются частоты слов в тексте, всего около 1500 признаков. Нужно автоматически определять категорию новых заявок. Каждую ночь модель должна дообучаться на новых заявках инкрементально — без хранения старых данных и без итеративной оптимизации. При этом модель должна быстро работать, потреблять мало памяти, а её решения должно быть легко объяснить. Какой метод выбрать и почему?

> **Эталон:** Наивный Байес (MultinomialNB) со сглаживанием Лапласа, так как он подходит для классификации текстов по частотам слов, хорошо работает на небольших выборках, быстро обучается и позволяет интерпретировать влияние слов на предсказание.

Рубрика / чек-лист:

```
Приемлемые методы:
- MultinomialNB со сглаживанием Лапласа
- BernoulliNB, если вместо частот слов использовать признаки их наличия или отсутствия
- ComplementNB (вариант наивного Байеса, устойчивый к несбалансированным классам)

Обязательные аргументы (все нужны для 2):
- размеченных данных мало, а признаков много, поэтому нужен метод, который может работать на
небольших выборках без сложного обучения
- MultinomialNB подходит для текстов, представленных частотами слов
- наивный Байес быстро обучается, требует относительно мало памяти и позволяет объяснять
предсказания через вероятности слов для разных классов
- наивный Байес дообучается инкрементально простым обновлением счётчиков слов по классам
(partial_fit), без хранения старых данных и без итеративной оптимизации

Типичные ошибки:
- выбор сложной нейросети без учёта малого количества размеченных данных и ограничений по
ресурсам
- выбор GaussianNB без объяснения, почему он подходит для частот слов
- утверждение, что наивный Байес требует большого количества данных для обучения
- отсутствие объяснения, почему выбранный метод подходит для текстовых признаков

2 — выбран подходящий вариант наивного Байеса (MultinomialNB, BernoulliNB, ComplementNB) и
```

```
приведены все обязательные аргументы
1 — выбран наивный Байес, но обоснование неполное или пропущены некоторые аргументы; ЛИБО
выбрана логистическая регрессия (в т. ч. SGDClassifier с partial_fit) с верными доводами, но
без учёта требования дообучаться без итеративной оптимизации
0 — выбран неподходящий метод (например, сложная нейросеть или kNN без оговорок про малый объём
и ресурсы) или отсутствует корректное обоснование
```

Ответ модели:

```
Для задачи с короткими текстами, 3 классами, ~1500 признаками (частоты слов) и требованиями
инкрементального дообучения без хранения старых данных и без итеративной оптимизации хорошо
подходит **наивный байесовский классификатор (Multinomial Naive Bayes)**.

Обоснование:

1. **Инкрементальность и отсутствие хранения старых данных.**
   Наивный Байес обучается путём простого накопления счётчиков: количество документов в каждом
классе и сумма частот каждого слова в каждом классе. Для дообучения достаточно обновить эти
счётчики на новых данных, не храня старые примеры и не запуская итеративные процедуры
оптимизации.

2. **Быстрота и малое потребление памяти.**
   Обучение и предсказание выполняются за линейное время от числа признаков и документов.
Память требуется только для хранения счётчиков по классам и словам, что при 1500 признаках и 3
классах крайне мало.

3. **Интерпретируемость.**
   Вклад каждого слова в решение легко объясняется через логарифмы вероятностей: можно
показать, какие слова сильнее всего повлияли на отнесение заявки к той или иной категории.

4. **Хорошая работа с высокоразмерными разреженными данными.**
   Частоты слов в коротких текстах дают сильно разреженную матрицу, а наивный Байес специально
хорошо приспособлен к таким данным и устойчив к проклятию размерности.

Альтернативы (линейные модели с SGD) тоже допускают инкрементальное обучение, но требуют
подбора гиперпараметров и менее стабильны при дообучении на малых порциях данных. Наивный Байес
проще и надёжнее в описанных условиях.

ОТВЕТ: Multinomial Naive Bayes
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C013 · тип C

Задача: Есть 20 000 записей телеметрии насосов (6 числовых признаков: давление, расход, температура и т. п.), разметки нет. Нужно выделить типичные режимы работы. Число режимов заранее неизвестно, в данных есть выбросы (аварийные замеры), а группы на графиках вытянутые и разной плотности. Какой метод выбрать и почему?

> **Эталон:** плотностная кластеризация HDBSCAN/OPTICS после масштабирования признаков

Рубрика / чек-лист:

```
Приемлемые методы:
– HDBSCAN или OPTICS (после масштабирования признаков)
– DBSCAN — на 2 балла только с обсуждением выбора eps/min_samples при разной плотности, иначе
не выше 1
– гауссова смесь (GMM) с полными ковариациями и выбором числа компонент по BIC — с оговоркой
про выбросы
```

```
Обязательные аргументы (все нужны для 2):
- число кластеров неизвестно: метод не должен требовать k заранее или должен выбирать его по критерию
- выбросы: плотностные методы помечают их как шум
- форма и плотность групп: k-means предполагает компактные кластеры близкого размера

Дополнительно (не обязательно):
- масштабирование признаков перед расчётом расстояний

Типичные ошибки:
- k-means с «методом локтя» без оговорок про форму кластеров и выбросы
- метод обучения с учителем (разметки нет)
- DBSCAN без обсуждения выбора eps/min_samples при разной плотности (HDBSCAN это решает)

2 — приемлемый метод + все три обязательных аргумента (неизвестное k, выбросы, форма/плотность)
1 — приемлемый метод, но пропущен обязательный аргумент; DBSCAN без обсуждения eps; k-means/GMM с осознанными оговорками
0 — метод с учителем, k-means без оговорок или нет обоснования
```

Ответ модели:

```
Для выделения типичных режимов работы насосов в условиях отсутствия разметки, неизвестного числа кластеров, наличия выбросов и групп различной формы и плотности, наиболее подходящим методом является DBSCAN (Density-Based Spatial Clustering of Applications with Noise).

Обоснование выбора:
- **Не требует предварительного знания количества кластеров**: DBSCAN автоматически определяет количество кластеров на основе плотности точек.
- **Эффективно обрабатывает выбросы**: точки, не принадлежащие ни одному плотному кластеру, помечаются как шум.
- **Способен находить кластеры произвольной формы**: подходит для вытянутых и неравномерно распределённых групп данных.
- **Хорошо работает с данными разной плотности**: адаптируется к локальной плотности данных.

ОТВЕТ: DBSCAN
УВЕРЕННОСТЬ: 90
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C014 · тип C

Задача: Есть 150 размеченных записей вибросигнала, 400 спектральных признаков, 3 класса состояния подшипника. Нужно интерпретируемое решение. Какой метод выбрать и почему?

> **Эталон:** регуляризованная линейная модель (логистическая регрессия с L1/ElasticNet) или неглубокое дерево после отбора признаков

Рубрика / чек-лист:

```
Приемлемые методы:
- мультиклассовая логистическая регрессия с L1 или ElasticNet-регуляризацией
- LDA с регуляризацией ковариации (shrinkage)
- неглубокое дерево решений после отбора или агрегации признаков (например, энергии по частотным полосам)
- линейный SVM с L1-регуляризацией и отбором признаков

Обязательные аргументы (все нужны для 2):
- признаков больше, чем объектов (400 > 150): высокий риск переобучения → регуляризация, отбор признаков или агрегация спектра
```

```
- интерпретируемость: веса отобранных частот или правила дерева можно объяснить инженеру

Дополнительно (не обязательно):
- оценка на стратифицированной кросс-валидации: данных мало, одна отложенная выборка будет
шумной

Типичные ошибки:
- нейросеть или бустинг без оговорок про малый объём и интерпретируемость
- kNN или SVM с RBF-ядром без снижения размерности и регуляризации
- PCA как главный шаг «для интерпретируемости» (главные компоненты трудно интерпретировать)
- одно разбиение train/test на 150 объектах без кросс-валидации

2 — приемлемый метод + оба обязательных аргумента (p > n → регуляризация/отбор;
интерпретируемость)
1 — приемлемый метод, но пропущен один обязательный аргумент; либо спорный метод с верными
аргументами
0 — неприемлемый метод или нет обоснования
```

Ответ модели:

```
Для интерпретации результата модели важно выбирать методы, которые позволяют понять вклад
каждого признака в итоговое решение. С учетом небольшого количества классов (3) и большого
числа признаков (400), хорошим выбором станет дерево решений или случайный лес.

Деревья решений обладают высокой интерпретируемостью благодаря наглядности структуры "если-то".
Они хорошо работают даже с большим числом признаков, автоматически выбирая наиболее значимые
признаки для разделения классов. Случайный лес улучшает точность дерева решений путем
ансамблевого обучения, сохраняя при этом возможность оценки важности признаков.

Однако учитывая требование максимальной интерпретируемости и простоты объяснения,
предпочтительнее выбрать именно одно дерево решений. Это позволит наглядно представить логику
классификации через структуру ветвления и легко объяснить результат конечным пользователям.

ОТВЕТ: Дерево решений
УВЕРЕННОСТЬ: 90
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C015 · тип C

Задача: На производстве нужно определять, исправно ли оборудование, по показаниям датчиков. Есть 2000 размеченных записей и 20 числовых признаков: температура, давление, вибрация и другие показатели. Из анализа данных видно, что классы нельзя хорошо разделить прямой линией. Модель должна быстро работать на обычном процессоре, не требовать много памяти. По регламенту сертификации допускается только одна модель — ансамбли (случайный лес, бустинг, стекинг) использовать нельзя. Подробно объяснять каждое предсказание не обязательно. Какой метод выбрать и почему?

> **Эталон:** SVM с RBF-ядром. Он позволяет строить нелинейную границу между классами и подходит для выборки такого размера. Признаки нужно масштабировать, а параметры C и gamma подобрать с помощью кросс-валидации.

Рубрика / чек-лист:

```
Приемлемые методы:
- SVM с RBF-ядром
- SVM с полиномиальным ядром при обоснованном выборе параметров

Обязательные аргументы (все нужны для 2):
```

```
– классы разделяются нелинейно, поэтому обычного линейного классификатора может быть
недостаточно
– SVM с нелинейным ядром позволяет построить более сложную границу между классами, не добавляя
новые признаки вручную
– размер выборки позволяет использовать ядерный SVM, а для корректной работы числовые признаки
нужно масштабировать

Типичные ошибки:
– выбор линейного SVM без учёта нелинейной границы между классами
– утверждение, что SVM может строить только линейные границы
– игнорирование необходимости масштабирования признаков
– утверждение, что SVM с RBF-ядром не требует настройки параметров C и gamma
– выбор случайного леса или бустинга вопреки запрету ансамблей в условии

2 – выбран подходящий SVM с нелинейным ядром и приведены все обязательные аргументы
1 – выбран SVM с нелинейным ядром, но обоснование неполное или пропущен один из обязательных
аргументов; ЛИБО выбрана допустимая одиночная альтернатива – kNN с масштабированием или
небольшая нейросеть (MLP) – с верными доводами про нелинейную границу и ресурсы
0 – выбран неподходящий метод (линейный классификатор без учёта нелинейной границы, ансамбль
вопреки запрету регламента) или нет корректного обоснования
```

Ответ модели:

```
Для данной задачи, где требуется классификация с неразделимыми классами, ограниченными
ресурсами и невозможностью использования ансамблевых методов, наиболее подходящим выбором будет
**логистическая регрессия**.

Логистическая регрессия – это простая и быстрая модель, которая хорошо работает на обычном
процессоре и не требует много памяти. Несмотря на то, что она строит линейную границу между
классами, она может быть адаптирована к сложным данным с помощью регуляризации (например, L1
или L2), что позволяет улучшить обобщающую способность модели. Также она имеет интерпретируемую
структуру, что полезно для задач, где важно понимать, какие признаки влияют на предсказание.

ОТВЕТ: Логистическая регрессия
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C016 · тип C

Задача: Есть 150 размеченных записей вибросигнала, 400 спектральных признаков, 3 класса состояния подшипника. Нужно интерпретируемое решение. Какой метод выбрать и почему?

> **Эталон:** регуляризованная линейная модель (логистическая регрессия с L1/ElasticNet) или неглубокое дерево после отбора признаков

Рубрика / чек-лист:

```
Приемлемые методы:
– мультиклассовая логистическая регрессия с L1 или ElasticNet-регуляризацией
– LDA с регуляризацией ковариации (shrinkage)
– неглубокое дерево решений после отбора или агрегации признаков (например, энергии по
частотным полосам)
– линейный SVM с L1-регуляризацией и отбором признаков

Обязательные аргументы (все нужны для 2):
– признаков больше, чем объектов (400 > 150): высокий риск переобучения → регуляризация, отбор
признаков или агрегация спектра
– интерпретируемость: веса отобранных частот или правила дерева можно объяснить инженеру
```

```
Дополнительно (не обязательно):
– оценка на стратифицированной кросс-валидации: данных мало, одна отложенная выборка будет
шумной

Типичные ошибки:
– нейросеть или бустинг без оговорок про малый объём и интерпретируемость
– kNN или SVM с RBF-ядром без снижения размерности и регуляризации
– PCA как главный шаг «для интерпретируемости» (главные компоненты трудно интерпретировать)
– одно разбиение train/test на 150 объектах без кросс-валидации

2 — приемлемый метод + оба обязательных аргумента (p > n → регуляризация/отбор;
интерпретируемость)
1 — приемлемый метод, но пропущен один обязательный аргумент; либо спорный метод с верными
аргументами
0 — неприемлемый метод или нет обоснования
```

Ответ модели:

```
Для задачи классификации состояния подшипника с ограниченным количеством размеченных данных
(150 записей) и большим количеством признаков (400), важно выбрать метод, который обеспечивает
интерпретируемость и способен работать с небольшими объемами данных.

Метод **логистическая регрессия** является хорошим выбором, так как он:
– Обладает хорошей интерпретируемостью (коэффициенты признаков показывают их влияние на
классификацию).
– Способен работать с высоким количеством признаков, особенно при использовании регуляризации
(например, L1-регуляризация).
– Не требует больших объемов данных для обучения, что подходит для данной задачи.
– Дает понятные результаты, что важно для интерпретируемого решения.

ОТВЕТ: Логистическая регрессия
УВЕРЕННОСТЬ: 90
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C017 · тип C

Задача: Есть 150 размеченных записей вибросигнала, 400 спектральных признаков, 3 класса состояния подшипника. Нужно интерпретируемое решение. Какой метод выбрать и почему?

> **Эталон:** регуляризованная линейная модель (логистическая регрессия с L1/ElasticNet) или неглубокое дерево после отбора признаков

Рубрика / чек-лист:

```
Приемлемые методы:
– мультиклассовая логистическая регрессия с L1 или ElasticNet-регуляризацией
– LDA с регуляризацией ковариации (shrinkage)
– неглубокое дерево решений после отбора или агрегации признаков (например, энергии по
частотным полосам)
– линейный SVM с L1-регуляризацией и отбором признаков

Обязательные аргументы (все нужны для 2):
– признаков больше, чем объектов (400 > 150): высокий риск переобучения → регуляризация, отбор
признаков или агрегация спектра
– интерпретируемость: веса отобранных частот или правила дерева можно объяснить инженеру

Дополнительно (не обязательно):
– оценка на стратифицированной кросс-валидации: данных мало, одна отложенная выборка будет
шумной
```

```
Типичные ошибки:
– нейросеть или бустинг без оговорок про малый объём и интерпретируемость
– kNN или SVM с RBF-ядром без снижения размерности и регуляризации
– PCA как главный шаг «для интерпретируемости» (главные компоненты трудно интерпретировать)
– одно разбиение train/test на 150 объектах без кросс-валидации

2 – приемлемый метод + оба обязательных аргумента (p > n → регуляризация/отбор;
интерпретируемость)
1 – приемлемый метод, но пропущен один обязательный аргумент; либо спорный метод с верными
аргументами
0 – неприемлемый метод или нет обоснования
```

Ответ модели:

```
В данной ситуации важно выбрать метод, который обеспечит не только хорошую предсказательную
способность, но и интерпретируемость модели. Учитывая относительно небольшой объём данных (150
записей) и большое количество признаков (400), есть риск переобучения при использовании сложных
моделей, таких как глубокие нейронные сети. В то же время метод должен быть способен улавливать
нелинейные зависимости между признаками и классами, которые могут присутствовать в данных
вибросигналов.

Одним из подходящих методов в этом случае является метод опорных векторов (SVM) с линейным
ядром или дерево решений. Однако дерево решений будет более интерпретируемым, так как его легко
визуализировать и понять логику принятия решений. Оно позволит увидеть, какие спектральные
признаки наиболее важны для классификации состояния подшипника, и как они влияют на итоговое
решение. При этом дерево решений не требует большого объёма данных для обучения и способно
работать с многомерными данными.

ОТВЕТ: дерево решений
УВЕРЕННОСТЬ: 85
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C018 · тип C

Задача: На промышленном предприятии система мониторинга собирает данные с 50 датчиков, установленных на производственном оборудовании. Датчики измеряют температуру, давление, вибрацию и энергопотребление. Накоплено 120 000 наблюдений, каждое описывается 50 числовыми признаками с разными единицами измерения. Предварительный анализ показал сильную корреляцию между многими признаками. Требуется уменьшить размерность данных для сокращения объёма хранения и ускорения последующего анализа, сохранив не менее 95% общей дисперсии. Меток классов нет. Физическая интерпретируемость каждого нового признака не обязательна, но важно минимизировать потери информации. Преобразование новых наблюдений должно выполняться быстро, а хранение преобразованных данных должно требовать меньше памяти. Какой метод выбрать и почему?

> **Эталон:** Метод главных компонент (PCA) после стандартизации числовых признаков. PCA строит ортогональные линейные комбинации исходных признаков, максимизирующие сохраняемую дисперсию. При сильной корреляции позволяет сократить размерность за счёт избыточной информации. Число компонент выбирается по накопленной доле объяснённой дисперсии не менее 95%. Стандартизация необходима из-за разных масштабов измерений. PCA не требует меток классов. Преобразование обучается на обучающей выборке и затем применяется к новым данным. Ограничения: ухудшение интерпретируемости признаков и потеря части информации; сохранение дисперсии не гарантирует сохранения всей информации, полезной для последующих задач.

Рубрика / чек-лист:

```
Приемлемые методы:
– PCA
– Principal Component Analysis
– Метод главных компонент
– StandardScaler + PCA
– IncrementalPCA при обоснованных ограничениях памяти
– TruncatedSVD на стандартизированных (центрированных) данных – математически совпадает с PCA

Обязательные аргументы (все нужны для 2):
– выбран PCA как метод снижения размерности числовых данных без использования меток классов
– объяснено, что сильная корреляция признаков позволяет сократить размерность за счёт избыточности информации
– указана необходимость стандартизации признаков перед PCA из-за различных единиц измерения и масштабов
– предложен выбор минимального числа компонент, обеспечивающего накопленную объяснённую дисперсию не менее 95%

Дополнительно (не обязательно):
– объяснено, что уменьшение числа признаков сокращает объём хранения преобразованных данных и может ускорить последующую обработку
– отмечено ограничение PCA: главные компоненты являются линейными комбинациями исходных признаков и хуже поддаются физической интерпретации
– указано, что параметры стандартизации и PCA обучаются только на обучающих данных, а к новым наблюдениям применяется уже построенное преобразование

Типичные ошибки:
– предложить обучение с учителем, несмотря на отсутствие меток классов
– применить PCA без стандартизации и не учесть различие масштабов признаков
– утверждать, что PCA просто выбирает несколько исходных признаков
– считать, что PCA гарантирует сохранение 95% информации при заранее фиксированном числе компонент
– утверждать, что PCA всегда улучшает качество последующей модели
– игнорировать потерю интерпретируемости главных компонент
– обучать PCA на всей выборке до разделения на обучающую и тестовую при последующей оценке модели

2 – выбран PCA, обоснованный вариант IncrementalPCA или TruncatedSVD на стандартизированных данных и раскрыты все четыре обязательных аргумента (без меток; корреляция → избыточность; стандартизация из-за разных единиц; число компонент по накопленной дисперсии ≥ 95%); дополнительные аргументы желательны, но не обязательны
1 – выбран PCA, основная идея снижения размерности объяснена правильно, но раскрыты не все обязательные аргументы либо есть отдельные неточности, не меняющие вывод
0 – предложено обучение с учителем, PCA описан как отбор исходных признаков или как классификатор, выбран неподходящий метод или нет обоснования
```

Ответ модели:

```
Для уменьшения размерности данных с сохранением не менее 95% общей дисперсии наиболее подходящим методом является **метод главных компонент** (Principal Component Analysis, PCA).

Обоснование выбора метода PCA:

– Задача требует снижения размерности большого набора признаков (50 измерений) с минимизацией потерь информации, что соответствует основной цели PCA – найти ортогональные компоненты, объясняющие наибольшую долю вариации данных.

– Отсутствие меток классов указывает на необходимость использования методов обучения без учителя, таких как PCA.

– Сильная корреляция между признаками подтверждает целесообразность применения PCA, поскольку этот метод эффективно устраняет избыточную информацию, вызванную мультиколлинеарностью переменных.
```

```
– Метод обеспечивает быстрое преобразование новых наблюдений благодаря линейным
преобразованиям, что удовлетворяет требованию скорости обработки.

– Компактное представление данных позволяет существенно сократить объём хранения, сохраняя
основную структуру исходной информации.

Таким образом, учитывая указанные требования и характеристики данных, оптимальным решением
является применение PCA.

ОТВЕТ: PCA
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C019 · тип C

Задача: Есть 20 000 записей телеметрии насосов (6 числовых признаков: давление, расход, температура и т. п.), разметки нет. Нужно выделить типичные режимы работы. Число режимов заранее неизвестно, в данных есть выбросы (аварийные замеры), а группы на графиках вытянутые и разной плотности. Какой метод выбрать и почему?

> **Эталон:** плотностная кластеризация HDBSCAN/OPTICS после масштабирования признаков

Рубрика / чек-лист:

```
Приемлемые методы:
– HDBSCAN или OPTICS (после масштабирования признаков)
– DBSCAN – на 2 балла только с обсуждением выбора eps/min_samples при разной плотности, иначе
не выше 1
– гауссова смесь (GMM) с полными ковариациями и выбором числа компонент по BIC – с оговоркой
про выбросы

Обязательные аргументы (все нужны для 2):
– число кластеров неизвестно: метод не должен требовать k заранее или должен выбирать его по
критерию
– выбросы: плотностные методы помечают их как шум
– форма и плотность групп: k-means предполагает компактные кластеры близкого размера

Дополнительно (не обязательно):
– масштабирование признаков перед расчётом расстояний

Типичные ошибки:
– k-means с «методом локтя» без оговорок про форму кластеров и выбросы
– метод обучения с учителем (разметки нет)
– DBSCAN без обсуждения выбора eps/min_samples при разной плотности (HDBSCAN это решает)

2 – приемлемый метод + все три обязательных аргумента (неизвестное k, выбросы, форма/плотность)
1 – приемлемый метод, но пропущен обязательный аргумент; DBSCAN без обсуждения eps; k-means/GMM
с осознанными оговорками
0 – метод с учителем, k-means без оговорок или нет обоснования
```

Ответ модели:

```
Для выделения типичных режимов работы без разметки и при наличии выбросов, а также при
вытянутых и разной плотности групп, наиболее подходящим методом является **кластеризация на
основе плотности (DBSCAN)**.

DBSCAN эффективно обрабатывает данные с разной плотностью, выявляет выбросы и не требует
заранее заданного числа кластеров. Он также хорошо справляется с шумом, что важно в данном
```

```
случае.

ОТВЕТ: DBSCAN
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C020 · тип C

Задача: Есть 20 000 записей телеметрии насосов (6 числовых признаков: давление, расход, температура и т. п.), разметки нет. Нужно выделить типичные режимы работы. Число режимов заранее неизвестно, в данных есть выбросы (аварийные замеры), а группы на графиках вытянутые и разной плотности. Какой метод выбрать и почему?

> **Эталон:** плотностная кластеризация HDBSCAN/OPTICS после масштабирования признаков

Рубрика / чек-лист:

```
Приемлемые методы:
– HDBSCAN или OPTICS (после масштабирования признаков)
– DBSCAN – на 2 балла только с обсуждением выбора eps/min_samples при разной плотности, иначе не выше 1
– гауссова смесь (GMM) с полными ковариациями и выбором числа компонент по BIC – с оговоркой про выбросы

Обязательные аргументы (все нужны для 2):
– число кластеров неизвестно: метод не должен требовать k заранее или должен выбирать его по критерию
– выбросы: плотностные методы помечают их как шум
– форма и плотность групп: k-means предполагает компактные кластеры близкого размера

Дополнительно (не обязательно):
– масштабирование признаков перед расчётом расстояний

Типичные ошибки:
– k-means с «методом локтя» без оговорок про форму кластеров и выбросы
– метод обучения с учителем (разметки нет)
– DBSCAN без обсуждения выбора eps/min_samples при разной плотности (HDBSCAN это решает)

2 – приемлемый метод + все три обязательных аргумента (неизвестное k, выбросы, форма/плотность)
1 – приемлемый метод, но пропущен обязательный аргумент; DBSCAN без обсуждения eps; k-means/GMM с осознанными оговорками
0 – метод с учителем, k-means без оговорок или нет обоснования
```

Ответ модели:

```
Для задачи выделения типичных режимов работы насосов без разметки подходит кластеризация.
Условия:
– число кластеров неизвестно;
– есть выбросы;
– кластеры вытянутые и разной плотности.

Метод DBSCAN хорошо справляется с выбросами и кластерами произвольной формы, но чувствителен к разной плотности и требует подбора eps.
Метод HDBSCAN лучше DBSCAN: он автоматически определяет кластеры разной плотности, устойчив к выбросам и не требует заранее задавать число кластеров.
Поэтому для данных условий наиболее подходящий выбор – HDBSCAN.
```

```
ОТВЕТ: HDBSCAN
УВЕРЕННОСТЬ: 92
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C021 · тип C

Задача: Нужно предсказывать брак партии на производственной линии. Есть 50 000 партий, 30 числовых параметров процесса и 10 категориальных (поставщик сырья, смена, линия), в части признаков пропуски, зависимости нелинейные, брак — 8% партий. Важна максимальная точность ранжирования (ROC AUC), интерпретируемость не обязательна, обучение на одном ноутбуке. Какой метод выбрать и почему?

> **Эталон:** градиентный бустинг над деревьями (CatBoost / LightGBM / HistGradientBoosting)

Рубрика / чек-лист:

```
Приемлемые методы:
– градиентный бустинг над деревьями: CatBoost, LightGBM, XGBoost,
HistGradientBoostingClassifier
– случайный лес — допустим с обоснованием (обычно уступает бустингу на таких данных)

Обязательные аргументы (все нужны для 2):
– табличные данные смешанного типа с нелинейностями — сильная сторона ансамблей деревьев
– пропуски и категориальные признаки: встроенная обработка (HistGB, CatBoost, LightGBM) или
явное кодирование
– дисбаланс 8%: метрика ROC AUC или PR AUC, стратифицированная кросс-валидация, при
необходимости веса классов

Дополнительно (не обязательно):
– подбор гиперпараметров (learning rate, глубина, число деревьев / early stopping) на кросс-
валидации

Типичные ошибки:
– глубокая нейросеть без обоснования для табличных данных такого размера
– kNN или SVM с RBF-ядром без обработки пропусков и категорий
– оценка по accuracy при 8% брака
– one-hot кодирование категорий с большим числом значений без оговорок

2 — бустинг (или лес с обоснованием) + все три обязательных аргумента (табличные смешанные
данные, пропуски/категории, дисбаланс)
1 — приемлемый метод, но пропущен обязательный аргумент; либо обоснование с ошибкой
0 — неприемлемый метод или нет обоснования
```

Ответ модели:

```
Для задачи с 50 000 наблюдений, смешанными типами признаков, пропусками, нелинейными
зависимостями и целью максимизации ROC AUC лучше всего подойдёт градиентный бустинг на решающих
деревьях, а именно **LightGBM** или **XGBoost**.

Обоснование:

1. **Нелинейность и взаимодействия** — деревья решений в ансамбле автоматически улавливают
сложные нелинейные зависимости и взаимодействия между числовыми и категориальными признаками
без ручного конструирования признаков.

2. **Смешанные типы данных** — градиентный бустинг нативно поддерживает категориальные признаки
(особенно LightGBM через `categorical_feature`), что избавляет от необходимости в one-hot
кодировании и снижает размерность.
```

```
3. **Пропуски** — деревья умеют обрабатывать пропуски без предварительного заполнения: при обучении пропуски направляются в ту ветвь, которая даёт наибольший прирост качества.

4. **Дисбаланс классов (8% брака)** — можно задать `scale_pos_weight` или использовать метрику AUC как оценочную, что позволяет модели лучше ранжировать редкий класс.

5. **Скорость и ресурсы** — LightGBM оптимизирован для работы на одном ноутбуке: гистограммный метод ускоряет обучение и снижает потребление памяти по сравнению с классическим GBM.

6. **Качество ранжирования** — градиентный бустинг обычно даёт один из лучших ROC AUC на табличных данных среднего размера, превосходя логистическую регрессию, случайный лес и SVM.

Альтернатива — CatBoost, который также хорошо работает с категориальными признаками и пропусками, но LightGBM обычно быстрее при сопоставимом качестве на таких объёмах данных.

ОТВЕТ: LightGBM (градиентный бустинг)
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C022 · тип C

Задача: Нужно предсказывать брак партии на производственной линии. Есть 50 000 партий, 30 числовых параметров процесса и 10 категориальных (поставщик сырья, смена, линия), в части признаков пропуски, зависимости нелинейные, брак — 8% партий. Важна максимальная точность ранжирования (ROC AUC), интерпретируемость не обязательна, обучение на одном ноутбуке. Какой метод выбрать и почему?

> **Эталон:** градиентный бустинг над деревьями (CatBoost / LightGBM / HistGradientBoosting)

Рубрика / чек-лист:

```
Приемлемые методы:
- градиентный бустинг над деревьями: CatBoost, LightGBM, XGBoost,
HistGradientBoostingClassifier
- случайный лес — допустим с обоснованием (обычно уступает бустингу на таких данных)

Обязательные аргументы (все нужны для 2):
- табличные данные смешанного типа с нелинейностями — сильная сторона ансамблей деревьев
- пропуски и категориальные признаки: встроенная обработка (HistGB, CatBoost, LightGBM) или
явное кодирование
- дисбаланс 8%: метрика ROC AUC или PR AUC, стратифицированная кросс-валидация, при
необходимости веса классов

Дополнительно (не обязательно):
- подбор гиперпараметров (learning rate, глубина, число деревьев / early stopping) на кросс-
валидации

Типичные ошибки:
- глубокая нейросеть без обоснования для табличных данных такого размера
- kNN или SVM с RBF-ядром без обработки пропусков и категорий
- оценка по accuracy при 8% брака
- one-hot кодирование категорий с большим числом значений без оговорок

2 — бустинг (или лес с обоснованием) + все три обязательных аргумента (табличные смешанные
данные, пропуски/категории, дисбаланс)
1 — приемлемый метод, но пропущен обязательный аргумент; либо обоснование с ошибкой
0 — неприемлемый метод или нет обоснования
```

Ответ модели:

```
Для задачи предсказания брака партии с высокой долей брака (8%) и необходимостью максимальной
точности ранжирования (ROC AUC) подходит метод, способный обрабатывать как числовые, так и
категориальные признаки, учитывать нелинейные зависимости и работать с пропусками.

**Gradient Boosting (например, XGBoost, LightGBM или CatBoost)** — оптимальный выбор.

**Причины:**
1. **Обработка пропусков:** CatBoost и LightGBM могут работать с пропусками без необходимости
их заполнения, что упрощает предобработку.
2. **Нелинейные зависимости:** Gradient Boosting эффективно учитывает сложные нелинейные
зависимости между признаками.
3. **Категориальные признаки:** CatBoost и LightGBM поддерживают категориальные признаки
напрямую, что упрощает работу с 10 категориальными признаками.
4. **Высокая точность ранжирования (ROC AUC):** Gradient Boosting показывает высокую
производительность в задачах бинарной классификации, особенно при наличии нелинейных
зависимостей и несбалансированных классов.
5. **Обучение на одном ноутбуке:** Все эти методы легко реализуются в Python с помощью
библиотек, таких как `xgboost`, `lightgbm` или `catboost`, и могут обучаться на одном ноутбуке
даже с 50 000 образцами.

ОТВЕТ: Gradient Boosting (CatBoost / XGBoost / LightGBM)
УВЕРЕННОСТЬ: 95
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C023 · тип C

Задача: В сервис поддержки поступают короткие текстовые заявки. Есть 300 размеченных заявок, которые относятся к 3 категориям: проблемы с оборудованием, сетью или ПО. В качестве признаков используются частоты слов в тексте, всего около 1500 признаков. Нужно автоматически определять категорию новых заявок. Каждую ночь модель должна дообучаться на новых заявках инкрементально — без хранения старых данных и без итеративной оптимизации. При этом модель должна быстро работать, потреблять мало памяти, а её решения должно быть легко объяснить. Какой метод выбрать и почему?

> **Эталон:** Наивный Байес (MultinomialNB) со сглаживанием Лапласа, так как он подходит для классификации текстов по частотам слов, хорошо работает на небольших выборках, быстро обучается и позволяет интерпретировать влияние слов на предсказание.

Рубрика / чек-лист:

```
Приемлемые методы:
– MultinomialNB со сглаживанием Лапласа
– BernoulliNB, если вместо частот слов использовать признаки их наличия или отсутствия
– ComplementNB (вариант наивного Байеса, устойчивый к несбалансированным классам)

Обязательные аргументы (все нужны для 2):
– размеченных данных мало, а признаков много, поэтому нужен метод, который может работать на
небольших выборках без сложного обучения
– MultinomialNB подходит для текстов, представленных частотами слов
– наивный Байес быстро обучается, требует относительно мало памяти и позволяет объяснять
предсказания через вероятности слов для разных классов
– наивный Байес дообучается инкрементально простым обновлением счётчиков слов по классам
(partial_fit), без хранения старых данных и без итеративной оптимизации

Типичные ошибки:
– выбор сложной нейросети без учёта малого количества размеченных данных и ограничений по
```

```
ресурсам
– выбор GaussianNB без объяснения, почему он подходит для частот слов
– утверждение, что наивный Байес требует большого количества данных для обучения
– отсутствие объяснения, почему выбранный метод подходит для текстовых признаков

2 — выбран подходящий вариант наивного Байеса (MultinomialNB, BernoulliNB, ComplementNB) и
приведены все обязательные аргументы
1 — выбран наивный Байес, но обоснование неполное или пропущены некоторые аргументы; ЛИБО
выбрана логистическая регрессия (в т. ч. SGDClassifier с partial_fit) с верными доводами, но
без учёта требования дообучаться без итеративной оптимизации
0 — выбран неподходящий метод (например, сложная нейросеть или kNN без оговорок про малый объём
и ресурсы) или отсутствует корректное обоснование
```

Ответ модели:

```
Для данной задачи оптимальным выбором является использование метода **Наивный Байес** (Naive
Bayes). Рассмотрим подробнее причины такого выбора:

1. **Инкрементальное обучение**: Метод Наивного Байеса поддерживает онлайн-обучение, что
позволяет обновлять модель на основе новых данных без необходимости хранить старые данные или
пересчитывать всю модель заново.

2. **Быстродействие и малое потребление памяти**: Наивный Байес имеет низкую вычислительную
сложность и требует небольшого объема памяти, что соответствует требованиям задачи.

3. **Интерпретируемость решений**: Решения модели Наивного Байеса основаны на вероятностном
подходе, что облегчает объяснение классификационных решений.

4. **Эффективность при большом количестве признаков**: Несмотря на большое количество признаков
(~1500 частот слов), Наивный Байес хорошо справляется благодаря предположению о независимости
признаков.

Таким образом, учитывая требования к инкрементальному обучению, быстродействию, низкому
потреблению ресурсов и интерпретируемости, Наивный Байес является наиболее подходящим методом
для данной задачи.

ОТВЕТ: Наивный Байес
УВЕРЕННОСТЬ: 90
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

C024 · тип C

Задача: На промышленном предприятии система мониторинга собирает данные с 50 датчиков, установленных на производственном оборудовании. Датчики измеряют температуру, давление, вибрацию и энергопотребление. Накоплено 120 000 наблюдений, каждое описывается 50 числовыми признаками с разными единицами измерения. Предварительный анализ показал сильную корреляцию между многими признаками. Требуется уменьшить размерность данных для сокращения объёма хранения и ускорения последующего анализа, сохранив не менее 95% общей дисперсии. Меток классов нет. Физическая интерпретируемость каждого нового признака не обязательна, но важно минимизировать потери информации. Преобразование новых наблюдений должно выполняться быстро, а хранение преобразованных данных должно требовать меньше памяти. Какой метод выбрать и почему?

> **Эталон:** Метод главных компонент (PCA) после стандартизации числовых признаков. PCA строит ортогональные линейные комбинации исходных признаков, максимизирующие сохраняемую дисперсию. При сильной корреляции позволяет сократить размерность за счёт

> избыточной информации. Число компонент выбирается по накопленной доле объяснённой дисперсии не менее 95%. Стандартизация необходима из-за разных масштабов измерений. PCA не требует меток классов. Преобразование обучается на обучающей выборке и затем применяется к новым данным. Ограничения: ухудшение интерпретируемости признаков и потеря части информации; сохранение дисперсии не гарантирует сохранения всей информации, полезной для последующих задач.

Рубрика / чек-лист:

```
Приемлемые методы:
– PCA
– Principal Component Analysis
– Метод главных компонент
– StandardScaler + PCA
– IncrementalPCA при обоснованных ограничениях памяти
– TruncatedSVD на стандартизированных (центрированных) данных — математически совпадает с PCA

Обязательные аргументы (все нужны для 2):
– выбран PCA как метод снижения размерности числовых данных без использования меток классов
– объяснено, что сильная корреляция признаков позволяет сократить размерность за счёт избыточности информации
– указана необходимость стандартизации признаков перед PCA из-за различных единиц измерения и масштабов
– предложен выбор минимального числа компонент, обеспечивающего накопленную объяснённую дисперсию не менее 95%

Дополнительно (не обязательно):
– объяснено, что уменьшение числа признаков сокращает объём хранения преобразованных данных и может ускорить последующую обработку
– отмечено ограничение PCA: главные компоненты являются линейными комбинациями исходных признаков и хуже поддаются физической интерпретации
– указано, что параметры стандартизации и PCA обучаются только на обучающих данных, а к новым наблюдениям применяется уже построенное преобразование

Типичные ошибки:
– предложить обучение с учителем, несмотря на отсутствие меток классов
– применить PCA без стандартизации и не учесть различие масштабов признаков
– утверждать, что PCA просто выбирает несколько исходных признаков
– считать, что PCA гарантирует сохранение 95% информации при заранее фиксированном числе компонент
– утверждать, что PCA всегда улучшает качество последующей модели
– игнорировать потерю интерпретируемости главных компонент
– обучать PCA на всей выборке до разделения на обучающую и тестовую при последующей оценке модели

2 — выбран PCA, обоснованный вариант IncrementalPCA или TruncatedSVD на стандартизированных данных и раскрыты все четыре обязательных аргумента (без меток; корреляция → избыточность; стандартизация из-за разных единиц; число компонент по накопленной дисперсии ≥ 95%); дополнительные аргументы желательны, но не обязательны
1 — выбран PCA, основная идея снижения размерности объяснена правильно, но раскрыты не все обязательные аргументы либо есть отдельные неточности, не меняющие вывод
0 — предложено обучение с учителем, PCA описан как отбор исходных признаков или как классификатор, выбран неподходящий метод или нет обоснования
```

Ответ модели:

```
Для решения задачи необходимо уменьшить размерность данных, сохранив не менее 95% общей дисперсии. Поскольку меток классов нет, методы, основанные на классификации, не подходят. Также важно минимизировать потери информации и обеспечить быстрое преобразование и хранение данных.

Идеальный выбор — **метод главных компонент (PCA)**. Он позволяет преобразовать данные в новое пространство, где каждая компонента отражает максимальную дисперсию данных. PCA сохраняет
```

```
большую часть информации, уменьшая размерность, что соответствует требованиям задачи. Кроме
того, PCA вычисляется быстро, и преобразованные данные требуют меньше памяти. Хотя новые
признаки не имеют физической интерпретируемости, это не требуется по условию задачи.

ОТВЕТ: PCA
УВЕРЕННОСТЬ: 100
```

Балл: □ 0 □ 1 □ 2
Комментарий: _____

D001 · тип D

Задача: Напиши код на Python (scikit-learn): на датасете load_breast_cancer подбери параметр C для SVM с RBF-ядром на кросс-валидации и выведи точность (accuracy) на отложенной тестовой выборке. В тест отложи 25% данных, random_state=42. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.979 ± 0.02 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Загрузка данных
data = load_breast_cancer()
X, y = data.data, data.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Пайплайн: стандартизация + SVM с RBF-ядром
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', random_state=42))
])

# Сетка параметров для подбора C
param_grid = {
    'svm__C': [0.01, 0.1, 1, 10, 100]
}

# Кросс-валидация для подбора C
```
````

```
grid_search = GridSearchCV(
    pipeline, param_grid, cv=5, scoring='accuracy', n_jobs=-1
)
grid_search.fit(X_train, y_train)

# Лучшая модель и предсказание на тестовой выборке
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

# Итоговая точность
accuracy = accuracy_score(y_test, y_pred)
print(accuracy)
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 95

Результат запуска кода:

```
код возврата 0, время 2.52 с, pass@1 = 1
stdout (конец):
0.9790209790209791
```

Метрика (автоматически): верно (авто): метрика 0.979021 в пределах ±0.02 (отн.) от 0.979

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D002 · тип D

Задача: Используя встроенный датасет load_wine из scikit-learn, реализуйте классификацию методом k ближайших соседей (kNN). Разделите данные на обучающую (75%) и тестовую (25%) выборки с random_state=42. Подберите оптимальное число соседей k из набора [1, 3, 5, 7, 9, 11, 13, 15] с помощью 5-блочной кросс-валидации на обучающей выборке, используя accuracy в качестве критерия выбора. Обучите модель с лучшим значением k на всей обучающей выборке и вычислите accuracy на тестовой выборке. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>, где число находится в диапазоне от 0 до 1.

> **Эталон:** эталонная метрика 0.9556 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
# Загрузка данных
data = load_wine()
X, y = data.data, data.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Подбор оптимального k с помощью кросс-валидации
k_values = [1, 3, 5, 7, 9, 11, 13, 15]
best_k = None
best_score = -1

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy')
    mean_score = np.mean(scores)
    if mean_score > best_score:
        best_score = mean_score
        best_k = k

# Обучение модели с лучшим k на всей обучающей выборке
best_knn = KNeighborsClassifier(n_neighbors=best_k)
best_knn.fit(X_train, y_train)

# Оценка на тестовой выборке
y_pred = best_knn.predict(X_test)
test_accuracy = accuracy_score(y_test, y_pred)

# Вывод итоговой метрики
print(f"METRIC: {test_accuracy:.4f}")
```
ОТВЕТ: код выше
УВЕРЕННОСТЬ: 95

Результат запуска кода:

```
код возврата 0, время 1.35 с, pass@1 = 1
stdout (конец):
METRIC: 0.7333
```

Метрика (автоматически): подсказка автопроверки — неверно: метрика 0.7333 вне ±0.05 (отн.) от 0.9556

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D003 · тип D

Задача: Используя встроенный датасет load_wine из scikit-learn, реализуйте классификацию методом k ближайших соседей (kNN). Разделите данные на обучающую (75%) и тестовую (25%) выборки с random_state=42. Подберите оптимальное число соседей k из набора [1, 3, 5, 7, 9, 11, 13, 15] с помощью 5-блочной кросс-валидации на обучающей выборке, используя accuracy в качестве критерия выбора. Обучите модель с лучшим значением k на всей обучающей выборке и вычислите accuracy на тестовой выборке. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>, где число находится в диапазоне от 0 до 1.

> **Эталон:** эталонная метрика 0.9556 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Загрузка данных
data = load_wine()
X, y = data.data, data.target

# Разделение данных на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Подбор оптимального числа соседей k
k_values = [1, 3, 5, 7, 9, 11, 13, 15]
best_k = None
best_score = 0

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy')
    mean_score = scores.mean()
    if mean_score > best_score:
        best_score = mean_score
        best_k = k

# Обучение модели с лучшим значением k на всей обучающей выборке
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)

# Вычисление accuracy на тестовой выборке
y_pred = knn_best.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

# Вывод итоговой метрики
print(f'METRIC: {accuracy}')
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
````

Результат запуска кода:

```
код возврата 0, время 1.25 с, pass@1 = 1
stdout (конец):
METRIC: 0.7777777777777778
```

Метрика (автоматически): подсказка автопроверки — неверно: метрика 0.777778 вне ±0.05 (отн.) от 0.9556

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D004 · тип D

Задача: Напиши код на Python (scikit-learn): сгенерируй данные make_classification(n_samples=5000, n_features=20, n_informative=5, weights=[0.95, 0.05], random_state=42). Отложи 25% в тест (random_state=42), подбери гиперпараметры RandomForestClassifier на кросс-валидации и выведи ROC AUC на тестовой выборке. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.8994 ± 0.03 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
import numpy as np

# Генерация данных
X, y = make_classification(n_samples=5000, n_features=20, n_informative=5,
                           weights=[0.95, 0.05], random_state=42)

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Определение параметров для поиска
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# Инициализация модели и GridSearchCV
rf = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='roc_auc', n_jobs=-1)

# Обучение модели
grid_search.fit(X_train, y_train)

# Получение лучшей модели
best_model = grid_search.best_estimator_

# Предсказание на тестовой выборке
````
````

```
y_pred_proba = best_model.predict_proba(X_test)[:, 1]

# Вычисление ROC AUC
roc_auc = roc_auc_score(y_test, y_pred_proba)

# Вывод результата
print(f"ROC AUC на тестовой выборке: {roc_auc:.4f}")
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 95

Результат запуска кода:

```
код возврата 0, время 15.76 с, pass@1 = 1
stdout (конец):
ROC AUC на тестовой выборке: 0.9051
```

Метрика (автоматически): верно (авто): метрика 0.9051 в пределах ±0.03 (отн.) от 0.8994

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D005 · тип D

Задача: Напиши код на Python (scikit-learn): сгенерируй данные make_classification(n_samples=5000, n_features=20, n_informative=5, weights=[0.95, 0.05], random_state=42). Отложи 25% в тест (random_state=42), подбери гиперпараметры RandomForestClassifier на кросс-валидации и выведи ROC AUC на тестовой выборке. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.8994 ± 0.03 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

# Генерация данных
X, y = make_classification(
    n_samples=5000,
    n_features=20,
    n_informative=5,
    weights=[0.95, 0.05],
    random_state=42
```

```
)

# Разделение на train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Подбор гиперпараметров RandomForestClassifier на кросс-валидации
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None]
}

rf = RandomForestClassifier(random_state=42, class_weight='balanced', n_jobs=-1)

grid_search = GridSearchCV(
    rf,
    param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    verbose=0
)

grid_search.fit(X_train, y_train)

# Лучшая модель и предсказание вероятностей
best_model = grid_search.best_estimator_
y_pred_proba = best_model.predict_proba(X_test)[:, 1]

# ROC AUC на тестовой выборке
roc_auc = roc_auc_score(y_test, y_pred_proba)
print(roc_auc)
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 95
```

**Результат запуска кода:**

```
код возврата 0, время 229.11 с, pass@1 = 1
stdout (конец):
0.9178725981251814
```

Метрика (автоматически): верно (авто): метрика 0.917873 в пределах ±0.03 (отн.) от 0.8994

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D006 · тип D

Задача: Используя встроенный датасет load_iris из scikit-learn, реализуйте кластеризацию методом KMeans без использования истинных меток классов. Подберите оптимальное число кластеров k из набора [2, 3, 4, 5, 6], максимизируя коэффициент силуэта (silhouette score). Перед кластеризацией стандартизируйте признаки (StandardScaler) и считайте силуэт в
```

стандартизированном пространстве. Для воспроизводимости используйте random_state=42 и n_init=10. После выбора оптимального k вычислите коэффициент силуэта полученной кластеризации на всех объектах датасета. Разделять данные на обучающую и тестовую выборки не требуется. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>.

> **Эталон:** эталонная метрика 0.5818 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Загрузка данных
iris = load_iris()
X = iris.data

# Стандартизация признаков
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Поиск оптимального числа кластеров
best_k = 0
best_score = -1
for k in [2, 3, 4, 5, 6]:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    if score > best_score:
        best_score = score
        best_k = k

# Кластеризация с оптимальным числом кластеров
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)
final_score = silhouette_score(X_scaled, labels)

# Вывод итоговой метрики
print(f"METRIC: {final_score}")
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
````

Результат запуска кода:

```
код возврата 0, время 1.24 с, pass@1 = 1
stdout (конец):
METRIC: 0.5817500491982808
```

Метрика (автоматически): верно (авто): метрика 0.58175 в пределах ±0.05 (отн.) от 0.5818

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D007 · тип D

Задача: Используя встроенный датасет load_iris из scikit-learn, реализуйте кластеризацию методом KMeans без использования истинных меток классов. Подберите оптимальное число кластеров k из набора [2, 3, 4, 5, 6], максимизируя коэффициент силуэта (silhouette score). Перед кластеризацией стандартизируйте признаки (StandardScaler) и считайте силуэт в стандартизированном пространстве. Для воспроизводимости используйте random_state=42 и n_init=10. После выбора оптимального k вычислите коэффициент силуэта полученной кластеризации на всех объектах датасета. Разделять данные на обучающую и тестовую выборки не требуется. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>.

> **Эталон:** эталонная метрика 0.5818 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
import numpy as np

# Загрузка данных
data = load_iris()
X = data.data

# Стандартизация признаков
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Определение возможных значений k
k_values = [2, 3, 4, 5, 6]
best_k = None
max_silhouette = -1

# Поиск оптимального числа кластеров
for k in k_values:
    # Кластеризация
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)

    # Вычисление коэффициента силуэта
````
````

```
    silhouette_avg = silhouette_score(X_scaled, labels)

    # Обновление лучшего результата
    if silhouette_avg > max_silhouette:
        max_silhouette = silhouette_avg
        best_k = k

# Финальная кластеризация с оптимальным числом кластеров
final_kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
final_labels = final_kmeans.fit_predict(X_scaled)

# Вычисление финальной метрики
final_silhouette = silhouette_score(X_scaled, final_labels)

# Печать результата
print(f"METRIC: {final_silhouette:.4f}")

# Ответ и уверенность
ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
```
```

**Результат запуска кода:**

```
код возврата 1, время 0.02 с, pass@1 = 0
извлечение кода: у кода убран общий отступ (блок был вложен в список)
stdout (конец):

stderr (конец):
File "/private/var/folders/fx/qskf3zh56ns2mvx4hbb9p8gc0000gn/T/d_exec_juj5bmq3/D6.py", line 45
    ОТВЕТ: код выше
               ^^^^
SyntaxError: invalid syntax
```

Метрика (автоматически): подсказка автопроверки — неверно: код упал (код возврата 1)

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D008 · тип D

Задача: Напиши код на Python (scikit-learn): на датасете load_digits построй модель «PCA, сохраняющий 95% дисперсии, + логистическая регрессия» и выведи accuracy на отложенной тестовой выборке. В тест отложи 25% данных, random_state=42. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.96 ± 0.02 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:
```

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score

# Загрузка данных
digits = load_digits()
X, y = digits.data, digits.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Создание пайплайна: PCA + Логистическая Регрессия
pipeline = Pipeline([
    ('pca', PCA(n_components=0.95)),  # Сохранить 95% дисперсии
    ('log_reg', LogisticRegression(max_iter=1000))
])

# Обучение модели
pipeline.fit(X_train, y_train)

# Предсказания на тестовой выборке
y_pred = pipeline.predict(X_test)

# Вычисление точности (accuracy)
accuracy = accuracy_score(y_test, y_pred)

# Печать результата
print(f"Accuracy: {accuracy:.4f}")
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100

Результат запуска кода:

```
код возврата 0, время 1.34 с, pass@1 = 1
stdout (конец):
Accuracy: 0.9667
```

Метрика (автоматически): верно (авто): метрика 0.9667 в пределах ±0.02 (отн.) от 0.96

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D009 · тип D

Задача: Используя встроенный датасет load_diabetes из scikit-learn, реализуйте прогнозирование количественного показателя прогрессирования диабета с помощью линейной регрессии с L2-регуляризацией (Ridge). Разделите данные на обучающую (80%) и тестовую (20%) выборки с random_state=42. Подберите оптимальное значение коэффициента регуляризации alpha из набора [0.01, 0.1, 1, 10, 100] с помощью 5-блочной кросс-валидации на обучающей выборке, минимизируя MAE. Обучите модель с лучшим значением alpha на всей обучающей выборке и

вычислите MAE на тестовой выборке. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>.

> **Эталон:** эталонная метрика 42.812 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error

# Загрузка данных
data = load_diabetes()
X, y = data.data, data.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Набор значений alpha
alphas = [0.01, 0.1, 1, 10, 100]

# Подбор оптимального alpha по кросс-валидации (MAE)
best_alpha = None
best_mae_cv = float('inf')

for alpha in alphas:
    model = Ridge(alpha=alpha)
    # Используем negative MAE, так как cross_val_score максимизирует метрику
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_absolute_error')
    mae_cv = -scores.mean()
    if mae_cv < best_mae_cv:
        best_mae_cv = mae_cv
        best_alpha = alpha

# Обучение модели с лучшим alpha на всей обучающей выборке
final_model = Ridge(alpha=best_alpha)
final_model.fit(X_train, y_train)

# Предсказание и оценка на тестовой выборке
y_pred = final_model.predict(X_test)
test_mae = mean_absolute_error(y_test, y_pred)

# Вывод метрики в требуемом формате
print(f"METRIC: {test_mae:.4f}")
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 95
````

Результат запуска кода:

```
код возврата 0, время 1.05 с, pass@1 = 1
stdout (конец):
METRIC: 42.8369
```

Метрика (автоматически): верно (авто): метрика 42.8369 в пределах ±0.05 (отн.) от 42.812

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D010 · тип D

Задача: Напиши код на Python (scikit-learn): на датасете load_digits построй модель «PCA, сохраняющий 95% дисперсии, + логистическая регрессия» и выведи accuracy на отложенной тестовой выборке. В тест отложи 25% данных, random_state=42. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.96 ± 0.02 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

```
```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score

# Загрузка данных
X, y = load_digits(return_X_y=True)

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Создание пайплайна: PCA (95% дисперсии) + логистическая регрессия
pipeline = Pipeline([
    ('pca', PCA(n_components=0.95, random_state=42)),
    ('logreg', LogisticRegression(max_iter=10000, random_state=42))
])

# Обучение модели
pipeline.fit(X_train, y_train)

# Предсказание и оценка точности
```
```

````
y_pred = pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(accuracy)
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 95
````

Результат запуска кода:

```
код возврата 0, время 1.04 с, pass@1 = 1
stdout (конец):
0.9511111111111111
```

Метрика (автоматически): верно (авто): метрика 0.951111 в пределах ±0.02 (отн.) от 0.96

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D011 · тип D

Задача: Используя встроенный датасет load_iris из scikit-learn, реализуйте кластеризацию методом KMeans без использования истинных меток классов. Подберите оптимальное число кластеров k из набора [2, 3, 4, 5, 6], максимизируя коэффициент силуэта (silhouette score). Перед кластеризацией стандартизируйте признаки (StandardScaler) и считайте силуэт в стандартизированном пространстве. Для воспроизводимости используйте random_state=42 и n_init=10. После выбора оптимального k вычислите коэффициент силуэта полученной кластеризации на всех объектах датасета. Разделять данные на обучающую и тестовую выборки не требуется. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>.

> **Эталон:** эталонная метрика 0.5818 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import numpy as np

# Загрузка датасета
iris = load_iris()
X = iris.data
````

```python
# Стандартизация признаков
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Перебор числа кластеров и выбор оптимального по коэффициенту силуэта
best_k = 2
best_silhouette = -1
for k in [2, 3, 4, 5, 6]:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    silhouette = silhouette_score(X_scaled, labels)
    if silhouette > best_silhouette:
        best_silhouette = silhouette
        best_k = k

# Кластеризация с оптимальным k
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# Вычисление итогового коэффициента силуэта
final_silhouette = silhouette_score(X_scaled, labels)

print(f"METRIC: {final_silhouette}")
```

ОТВЕТ:
```python
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import numpy as np

# Загрузка датасета
iris = load_iris()
X = iris.data

# Стандартизация признаков
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Перебор числа кластеров и выбор оптимального по коэффициенту силуэта
best_k = 2
best_silhouette = -1
for k in [2, 3, 4, 5, 6]:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    silhouette = silhouette_score(X_scaled, labels)
    if silhouette > best_silhouette:
        best_silhouette = silhouette
        best_k = k

# Кластеризация с оптимальным k
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# Вычисление итогового коэффициента силуэта
final_silhouette = silhouette_score(X_scaled, labels)

print(f"METRIC: {final_silhouette}")
```
УВЕРЕННОСТЬ: 100

Результат запуска кода:

```
код возврата 0, время 1.17 с, pass@1 = 1
извлечение кода: блоков кода 2, взят самый длинный
stdout (конец):
METRIC: 0.5817500491982808
```

Метрика (автоматически): верно (авто): метрика 0.58175 в пределах ±0.05 (отн.) от 0.5818

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D012 · тип D

Задача: Используя встроенный датасет load_wine из scikit-learn, реализуйте классификацию методом k ближайших соседей (kNN). Разделите данные на обучающую (75%) и тестовую (25%) выборки с random_state=42. Подберите оптимальное число соседей k из набора [1, 3, 5, 7, 9, 11, 13, 15] с помощью 5-блочной кросс-валидации на обучающей выборке, используя accuracy в качестве критерия выбора. Обучите модель с лучшим значением k на всей обучающей выборке и вычислите accuracy на тестовой выборке. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>, где число находится в диапазоне от 0 до 1.

> **Эталон:** эталонная метрика 0.9556 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# Загрузка датасета
wine = load_wine()
X, y = wine.data, wine.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Подбор оптимального k с помощью 5-блочной кросс-валидации
best_k = 1
best_score = 0
for k in [1, 3, 5, 7, 9, 11, 13, 15]:
    scores = cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train, y_train, cv=5,
scoring='accuracy')
    mean_score = np.mean(scores)
    if mean_score > best_score:
        best_score = mean_score
        best_k = k
````
````

```
# Обучение модели с лучшим k на всей обучающей выборке
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train, y_train)

# Вычисление accuracy на тестовой выборке
test_accuracy = knn.score(X_test, y_test)
print(f"METRIC: {test_accuracy:.1f}")
```

ОТВЕТ:
```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# Загрузка датасета
wine = load_wine()
X, y = wine.data, wine.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Подбор оптимального k с помощью 5-блочной кросс-валидации
best_k = 1
best_score = 0
for k in [1, 3, 5, 7, 9, 11, 13, 15]:
    scores = cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train, y_train, cv=5,
scoring='accuracy')
    mean_score = np.mean(scores)
    if mean_score > best_score:
        best_score = mean
        best_k = k

# Обучение модели с лучшим k на всей обучающей выборке
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train, y_train)

# Вычисление accuracy на тестовой выборке
test_accuracy = knn.score(X_test, y_test)
print(f"METRIC: {test_accuracy:.1f}")
```
УВЕРЕННОСТЬ: 95

Результат запуска кода:

```
код возврата 0, время 1.29 с, pass@1 = 1
извлечение кода: блоков кода 2, взят самый длинный
stdout (конец):
METRIC: 0.8
```

Метрика (автоматически): подсказка автопроверки — неверно: метрика 0.8 вне ±0.05 (отн.) от 0.9556

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D013 · тип D

Задача: Напиши код на Python (scikit-learn): на датасете load_breast_cancer подбери параметр C для SVM с RBF-ядром на кросс-валидации и выведи точность (accuracy) на отложенной тестовой выборке. В тест отложи 25% данных, random_state=42. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.979 ± 0.02 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn import datasets
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Загрузка датасета
data = datasets.load_breast_cancer()
X, y = data.data, data.target

# Разделение данных на тренировочную и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Настройка SVM с RBF-ядром и подбор параметра C через кросс-валидацию
param_grid = {'C': [0.1, 1, 10, 100]}
grid_search = GridSearchCV(SVC(kernel='rbf'), param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# Определение лучшего значения C и обучение модели с этим параметром
best_C = grid_search.best_params_['C']
best_model = SVC(kernel='rbf', C=best_C)
best_model.fit(X_train, y_train)

# Оценка точности на тестовой выборке
y_pred = best_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

# Вывод точности
print(f'Точность на тестовой выборке: {accuracy:.4f}')
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
````

Результат запуска кода:

```
код возврата 0, время 1.01 с, pass@1 = 1
stdout (конец):
Точность на тестовой выборке: 0.9860
```

Метрика (автоматически): верно (авто): метрика 0.986 в пределах ±0.02 (отн.) от 0.979

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D014 · тип D

Задача: Используя встроенный датасет load_wine из scikit-learn, реализуйте классификацию методом k ближайших соседей (kNN). Разделите данные на обучающую (75%) и тестовую (25%) выборки с random_state=42. Подберите оптимальное число соседей k из набора [1, 3, 5, 7, 9, 11, 13, 15] с помощью 5-блочной кросс-валидации на обучающей выборке, используя accuracy в качестве критерия выбора. Обучите модель с лучшим значением k на всей обучающей выборке и вычислите accuracy на тестовой выборке. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>, где число находится в диапазоне от 0 до 1.

> **Эталон:** эталонная метрика 0.9556 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# Загрузка данных
data = load_wine()
X, y = data.data, data.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Определение возможных значений k
k_values = [1, 3, 5, 7, 9, 11, 13, 15]
best_k = None
best_accuracy = 0

# Поиск лучшего значения k через кросс-валидацию
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy')
    mean_accuracy = np.mean(scores)

    if mean_accuracy > best_accuracy:
        best_accuracy = mean_accuracy
        best_k = k

# Обучение модели с оптимальным k на всей обучающей выборке
best_model = KNeighborsClassifier(n_neighbors=best_k)
best_model.fit(X_train, y_train)
````
````

```
# Вычисление точности на тестовой выборке
test_accuracy = best_model.score(X_test, y_test)

# Печать результата
print(f'METRIC: {test_accuracy}')
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100

Результат запуска кода:

```
код возврата 0, время 1.24 с, pass@1 = 1
извлечение кода: у кода убран общий отступ (блок был вложен в список)
stdout (конец):
METRIC: 0.7777777777777778
```

Метрика (автоматически): подсказка автопроверки — неверно: метрика 0.777778 вне ±0.05 (отн.) от 0.9556

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D015 · тип D

Задача: Используя встроенный датасет load_diabetes из scikit-learn, реализуйте прогнозирование количественного показателя прогрессирования диабета с помощью линейной регрессии с L2-регуляризацией (Ridge). Разделите данные на обучающую (80%) и тестовую (20%) выборки с random_state=42. Подберите оптимальное значение коэффициента регуляризации alpha из набора [0.01, 0.1, 1, 10, 100] с помощью 5-блочной кросс-валидации на обучающей выборке, минимизируя MAE. Обучите модель с лучшим значением alpha на всей обучающей выборке и вычислите MAE на тестовой выборке. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>.

> **Эталон:** эталонная метрика 42.812 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error

# Загрузка данных
diabetes = load_diabetes()
````
````

```
X, y = diabetes.data, diabetes.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Определение списка альфа-значений для подбора
alphas = [0.01, 0.1, 1, 10, 100]
best_alpha = None
best_mae = float('inf')

# Поиск оптимального значения alpha через кросс-валидацию
for alpha in alphas:
    ridge = Ridge(alpha=alpha)
    maes = -cross_val_score(ridge, X_train, y_train, cv=5, scoring='neg_mean_absolute_error')
    avg_mae = maes.mean()

    if avg_mae < best_mae:
        best_mae = avg_mae
        best_alpha = alpha

# Обучение модели с оптимальным alpha на всей обучающей выборке
final_ridge = Ridge(alpha=best_alpha)
final_ridge.fit(X_train, y_train)

# Вычисление MAE на тестовой выборке
y_pred = final_ridge.predict(X_test)
test_mae = mean_absolute_error(y_test, y_pred)

# Печать результата
print(f"METRIC: {test_mae:.3f}")
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
```

**Результат запуска кода:**

```
код возврата 0, время 0.98 с, pass@1 = 1
извлечение кода: у кода убран общий отступ (блок был вложен в список)
stdout (конец):
METRIC: 42.837
```

Метрика (автоматически): верно (авто): метрика 42.837 в пределах ±0.05 (отн.) от 42.812

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D016 · тип D

Задача: Напиши код на Python (scikit-learn): сгенерируй данные make_classification(n_samples=5000, n_features=20, n_informative=5, weights=[0.95, 0.05], random_state=42). Отложи 25% в тест (random_state=42), подбери гиперпараметры RandomForestClassifier на кросс-валидации и выведи ROC AUC на тестовой выборке. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.8994 ± 0.03 (отн.)

Рубрика / чек-лист:
```

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import roc_auc_score
from sklearn.ensemble import RandomForestClassifier

# Генерация синтетических данных
X, y = make_classification(
    n_samples=5000,
    n_features=20,
    n_informative=5,
    weights=[0.95, 0.05],
    random_state=42
)

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Подбор параметров модели через кросс-валидацию
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
}

grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    scoring='roc_auc',
    cv=5,
    verbose=1,
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

# Лучшая модель
best_model = grid_search.best_estimator_

# Предсказания вероятностей класса 1
y_pred_proba = best_model.predict_proba(X_test)[:, 1]

# Вычисление ROC-AUC на тестовой выборке
roc_auc = roc_auc_score(y_test, y_pred_proba)

# Печать результата
print(f'ROC-AUC на тестовой выборке: {roc_auc:.4f}')
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
````

Результат запуска кода:

```
код возврата 0, время 22.33 с, pass@1 = 1
stdout (конец):
Fitting 5 folds for each of 27 candidates, totalling 135 fits
ROC-AUC на тестовой выборке: 0.9014
```

Метрика (автоматически): верно (авто): метрика 0.9014 в пределах ±0.03 (отн.) от 0.8994

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D017 · тип D

Задача: Напиши код на Python (scikit-learn): на датасете load_digits построй модель «PCA, сохраняющий 95% дисперсии, + логистическая регрессия» и выведи accuracy на отложенной тестовой выборке. В тест отложи 25% данных, random_state=42. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.96 ± 0.02 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score

# Загрузка данных
digits = load_digits()
X, y = digits.data, digits.target

# Разделение данных на тренировочную и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Создание пайплайна: PCA + логистическая регрессия
pipeline = make_pipeline(PCA(n_components=0.95), LogisticRegression(max_iter=10000,
random_state=42))

# Обучение модели
pipeline.fit(X_train, y_train)

# Предсказание на тестовой выборке
y_pred = pipeline.predict(X_test)

# Вычисление accuracy
```
````

```
accuracy = accuracy_score(y_test, y_pred)

# Вывод результата
print(f"Accuracy: {accuracy}")
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100

Результат запуска кода:

```
код возврата 0, время 1.05 с, pass@1 = 1
stdout (конец):
Accuracy: 0.9666666666666667
```

Метрика (автоматически): верно (авто): метрика 0.966667 в пределах ±0.02 (отн.) от 0.96

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D018 · тип D

Задача: Напиши код на Python (scikit-learn): на датасете load_breast_cancer подбери параметр C для SVM с RBF-ядром на кросс-валидации и выведи точность (accuracy) на отложенной тестовой выборке. В тест отложи 25% данных, random_state=42. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.979 ± 0.02 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Загружаем датасет
data = datasets.load_breast_cancer()
X, y = data.data, data.target

# Делим на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Создаем модель SVM с RBF-ядром
model = SVC(kernel='rbf')

# Подбираем параметр C через кросс-валидацию
param_grid = {'C': [0.1, 1, 10, 100]}
```

```
grid_search = GridSearchCV(model, param_grid, cv=5)
grid_search.fit(X_train, y_train)

# Лучшая найденная модель
best_model = grid_search.best_estimator_

# Предсказания на тестовой выборке
y_pred = best_model.predict(X_test)

# Вычисляем точность модели
accuracy = accuracy_score(y_test, y_pred)

# Печатаем результат
print(f"Точность модели на тестовой выборке: {accuracy:.3f}")

# Ответ и уверенность
ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
```

Результат запуска кода:

```
код возврата 1, время 0.02 с, pass@1 = 0
stdout (конец):

stderr (конец):
File "/private/var/folders/fx/qskf3zh56ns2mvx4hbb9p8gc0000gn/T/d_exec_khk2bva8/D1.py", line 34
    ОТВЕТ: код выше
               ^^^^
SyntaxError: invalid syntax
```

Метрика (автоматически): подсказка автопроверки — неверно: код упал (код возврата 1)

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D019 · тип D

Задача: Напиши код на Python (scikit-learn): на датасете load_breast_cancer подбери параметр C для SVM с RBF-ядром на кросс-валидации и выведи точность (accuracy) на отложенной тестовой выборке. В тест отложи 25% данных, random_state=42. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.979 ± 0.02 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

```python
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import numpy as np

# Загрузка датасета
data = load_breast_cancer()
X, y = data.data, data.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Определение параметров для поиска
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto']
}

# Создание модели SVM с RBF-ядром
svm = SVC(kernel='rbf')

# Кросс-валидация для подбора параметра C
grid_search = GridSearchCV(svm, param_grid, cv=5)
grid_search.fit(X_train, y_train)

# Получение лучшей модели
best_model = grid_search.best_estimator_

# Предсказание на тестовой выборке
y_pred = best_model.predict(X_test)

# Вычисление и вывод точности
accuracy = accuracy_score(y_test, y_pred)
print(f"Итоговая точность (accuracy): {accuracy:.4f}")
```

ОТВЕТ:
```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import numpy as np

# Загрузка датасета
data = load_breast_cancer()
X, y = data.data, data.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Определение параметров для поиска
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto']
}

# Создание модели SVM с RBF-ядром
svm = SVC(kernel='rbf')

# Кросс-валидация для подбора параметра C
grid_search = GridSearchCV(svm, param_grid, cv=5)
grid_search.fit(X_train, y_train)
```

```
# Получение лучшей модели
best_model = grid_search.best_estimator_

# Предсказание на тестовой выборке
y_pred = best_model.predict(X_test)

# Вычисление и вывод точности
accuracy = accuracy_score(y_test, y_pred)
print(f"Итоговая точность (accuracy): {accuracy:.4f}")
```
УВЕРЕННОСТЬ: 100
```

**Результат запуска кода:**

```
код возврата 0, время 1.18 с, pass@1 = 1
извлечение кода: блоков кода 2, взят самый длинный
stdout (конец):
Итоговая точность (accuracy): 0.9860
```

Метрика (автоматически): верно (авто): метрика 0.986 в пределах ±0.02 (отн.) от 0.979

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D020 · тип D

Задача: Напиши код на Python (scikit-learn): на датасете load_digits построй модель «PCA, сохраняющий 95% дисперсии, + логистическая регрессия» и выведи accuracy на отложенной тестовой выборке. В тест отложи 25% данных, random_state=42. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.96 ± 0.02 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

```
```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import numpy as np

# Загрузка датасета
digits = load_digits()
X, y = digits.data, digits.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

```
# Применение PCA для сохранения 95% дисперсии
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

# Обучение логистической регрессии
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_pca, y_train)

# Предсказание на тестовой выборке
y_pred = log_reg.predict(X_test_pca)

# Вычисление accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
```

```
ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
```

Результат запуска кода:

```
код возврата 0, время 1.13 с, pass@1 = 1
stdout (конец):
Accuracy: 0.9667
```

Метрика (автоматически): верно (авто): метрика 0.9667 в пределах ±0.02 (отн.) от 0.96

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D021 · тип D

Задача: Используя встроенный датасет load_diabetes из scikit-learn, реализуйте прогнозирование количественного показателя прогрессирования диабета с помощью линейной регрессии с L2-регуляризацией (Ridge). Разделите данные на обучающую (80%) и тестовую (20%) выборки с random_state=42. Подберите оптимальное значение коэффициента регуляризации alpha из набора [0.01, 0.1, 1, 10, 100] с помощью 5-блочной кросс-валидации на обучающей выборке, минимизируя MAE. Обучите модель с лучшим значением alpha на всей обучающей выборке и вычислите MAE на тестовой выборке. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>.

> **Эталон:** эталонная метрика 42.812 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error
import numpy as np

# Загрузка датасета
diabetes = load_diabetes()
X, y = diabetes.data, diabetes.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Определение параметров для поиска
param_grid = {'alpha': [0.01, 0.1, 1, 10, 100]}

# 5-блочная кросс-валидация для подбора alpha
grid_search = GridSearchCV(Ridge(), param_grid, scoring='neg_mean_absolute_error', cv=5)
grid_search.fit(X_train, y_train)

# Получение лучшего alpha
best_alpha = grid_search.best_params_['alpha']
best_model = Ridge(alpha=best_alpha)
best_model.fit(X_train, y_train)

# Вычисление MAE на тестовой выборке
y_pred = best_model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)

print(f"METRIC: {mae}")
```

ОТВЕТ:
```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error
import numpy as np

# Загрузка датасета
diabetes = load_diabetes()
X, y = diabetes.data, diabetes.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Определение параметров для поиска
param_grid = {'alpha': [0.01, 0.1, 1, 10, 100]}

# 5-блочная кросс-валидация для подбора alpha
grid_search = GridSearchCV(Ridge(), param_grid, scoring='neg_mean_absolute_error', cv=5)
grid_search.fit(X_train, y_train)

# Получение лучшего alpha
best_alpha = grid_search.best_params_['alpha']
best_model = Ridge(alpha=best_alpha)
best_model.fit(X_train, y_train)

# Вычисление MAE на тестовой выборке
y_pred = best_model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
```

```
print(f"METRIC: {mae}")
```
УВЕРЕННОСТЬ: 100

Результат запуска кода:

```
код возврата 0, время 1.01 с, pass@1 = 1
извлечение кода: блоков кода 2, взят самый длинный
stdout (конец):
METRIC: 42.836887676091756
```

Метрика (автоматически): верно (авто): метрика 42.8369 в пределах ±0.05 (отн.) от 42.812

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D022 · тип D

Задача: Напиши код на Python (scikit-learn): сгенерируй данные make_classification(n_samples=5000, n_features=20, n_informative=5, weights=[0.95, 0.05], random_state=42). Отложи 25% в тест (random_state=42), подбери гиперпараметры RandomForestClassifier на кросс-валидации и выведи ROC AUC на тестовой выборке. Код должен напечатать итоговое значение метрики.

> **Эталон:** эталонная метрика 0.8994 ± 0.03 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

```python
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

# Генерация данных
X, y = make_classification(n_samples=5000, n_features=20, n_informative=5, weights=[0.95,
0.05], random_state=42)

# Разделение данных на тренировочную и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Настройка гиперпараметров с помощью GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10]
}
```

```
grid_search = GridSearchCV(estimator=RandomForestClassifier(random_state=42),
param_grid=param_grid, cv=5, scoring='roc_auc', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Получение лучшего классификатора
best_clf = grid_search.best_estimator_

# Прогнозирование на тестовой выборке
y_pred_proba = best_clf.predict_proba(X_test)[:, 1]

# Расчет ROC AUC
roc_auc = roc_auc_score(y_test, y_pred_proba)

# Вывод результата
print(f'ROC AUC на тестовой выборке: {roc_auc}')
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
```

**Результат запуска кода:**

```
код возврата 0, время 17.55 с, pass@1 = 1
stdout (конец):
ROC AUC на тестовой выборке: 0.9014773974590278
```

Метрика (автоматически): верно (авто): метрика 0.901477 в пределах ±0.03 (отн.) от 0.8994

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D023 · тип D

Задача: Используя встроенный датасет load_diabetes из scikit-learn, реализуйте прогнозирование количественного показателя прогрессирования диабета с помощью линейной регрессии с L2-регуляризацией (Ridge). Разделите данные на обучающую (80%) и тестовую (20%) выборки с random_state=42. Подберите оптимальное значение коэффициента регуляризации alpha из набора [0.01, 0.1, 1, 10, 100] с помощью 5-блочной кросс-валидации на обучающей выборке, минимизируя MAE. Обучите модель с лучшим значением alpha на всей обучающей выборке и вычислите MAE на тестовой выборке. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>.

> **Эталон:** эталонная метрика 42.812 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

```
```python
from sklearn.datasets import load_diabetes
```

```
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error
import numpy as np

# Загрузка данных
data = load_diabetes()
X, y = data.data, data.target

# Разделение данных на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Определение набора значений alpha для поиска оптимального
alphas = [0.01, 0.1, 1, 10, 100]

# Инициализация объекта для 5-блочной кросс-валидации
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Поиск оптимального alpha
best_alpha = None
best_mae = np.inf
for alpha in alphas:
    model = Ridge(alpha=alpha)
    mae_scores = -cross_val_score(model, X_train, y_train, cv=kf,
scoring='neg_mean_absolute_error')
    mean_mae = np.mean(mae_scores)
    if mean_mae < best_mae:
        best_mae = mean_mae
        best_alpha = alpha

# Обучение модели с лучшим alpha на всей обучающей выборке
best_model = Ridge(alpha=best_alpha)
best_model.fit(X_train, y_train)

# Прогноз на тестовой выборке и вычисление MAE
y_pred = best_model.predict(X_test)
test_mae = mean_absolute_error(y_test, y_pred)

# Вывод итоговой метрики
print(f'METRIC: {test_mae}')
```

ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100

Результат запуска кода:

```
код возврата 0, время 1.02 с, pass@1 = 1
stdout (конец):
METRIC: 42.836887676091756
```

Метрика (автоматически): верно (авто): метрика 42.8369 в пределах ±0.05 (отн.) от 42.812

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п
Результат верен (если не определён автоматически): □ да □ нет
Комментарий: _____

D024 · тип D

Задача: Используя встроенный датасет load_iris из scikit-learn, реализуйте кластеризацию методом KMeans без использования истинных меток классов. Подберите оптимальное число кластеров k из набора [2, 3, 4, 5, 6], максимизируя коэффициент силуэта (silhouette score). Перед кластеризацией стандартизируйте признаки (StandardScaler) и считайте силуэт в стандартизированном пространстве. Для воспроизводимости используйте random_state=42 и n_init=10. После выбора оптимального k вычислите коэффициент силуэта полученной кластеризации на всех объектах датасета. Разделять данные на обучающую и тестовую выборки не требуется. Выведите итоговую метрику в последней строке строго в формате METRIC: <число>.

> **Эталон:** эталонная метрика 0.5818 ± 0.05 (отн.)

Рубрика / чек-лист:

```
Чек-лист (data/checklist_D.md): 1 — ошибка есть, 0 — нет, н/п — неприменимо.
1. масштабирование/препроцессинг до разбиения
2. подбор гиперпараметров по тесту
3. оценка на обучающей выборке
4. accuracy при сильном дисбалансе
5. нет стратификации
6. не зафиксирован random_state
```

Ответ модели:

````
```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Загрузка данных
iris = load_iris()
X = iris.data

# Стандартизация признаков
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Подбор оптимального k по коэффициенту силуэта
k_values = [2, 3, 4, 5, 6]
best_k = None
best_silhouette = -1

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    sil_score = silhouette_score(X_scaled, labels)
    if sil_score > best_silhouette:
        best_silhouette = sil_score
        best_k = k

# Итоговая кластеризация с оптимальным k
kmeans_final = KMeans(n_clusters=best_k, random_state=42, n_init=10)
final_labels = kmeans_final.fit_predict(X_scaled)
final_silhouette = silhouette_score(X_scaled, final_labels)

print(f"METRIC: {final_silhouette:.6f}")
```
````

```
ОТВЕТ: код выше
УВЕРЕННОСТЬ: 100
```

Результат запуска кода:

```
код возврата 0, время 1.17 с, pass@1 = 1
stdout (конец):
METRIC: 0.581750
```

Метрика (автоматически): верно (авто): метрика 0.58175 в пределах ±0.05 (отн.) от 0.5818

Чек-лист (1 — ошибка есть, 0 — нет, н/п): 1) □0 □1 □н/п 2) □0 □1 □н/п 3) □0 □1 □н/п 4) □0 □1 □н/п 5) □0 □1 □н/п 6) □0 □1 □н/п

Результат верен (если не определён автоматически): □ да □ нет

Комментарий: _____